

《数字信号处理》实验指导

目录

目录	I
第 1 章 MATLAB简介	1
1.1 MATLAB软件在信号与系统中的应用介绍	1
1.2 MATLAB软件使用入门	1
第 2 章 离散时间信号	16
2.1 实验原理及实例分析	16
第 3 章 离散时间LTI系统的时域分析	24
3.1 实验原理及实例分析	24
第 4 章 z变换及离散时间LTI系统的z域分析	30
4.1 实验原理及实例分析	30
第 5 章 离散傅立叶变换及其快速算法	41
5.1 信号变换概述	错误！未定义书签。
5.2 离散傅立叶变换	41
5.3 快速傅立叶变换（FFT）	49
第 6 章 IIR数字滤波器	56
6.1 实验原理及实例分析	56
第 7 章 FIR数字滤波器	63
7.1 实验原理及实例分析	63

第 1 章 MATLAB 简介

1.1 MATLAB 软件在信号与系统中的应用介绍

MATLAB 的名称源自 Matrix Laboratory，1984 年由美国 Mathworks 公司推向市场。它是一种科学计算软件，专门以矩阵的形式处理数据。MATLAB 将高性能的数值计算和可视化集成在一起，并提供了大量的内置函数，从而被广泛地应用于科学计算、控制系统、信息处理等领域的分析、仿真和设计工作。1993 年 MathWorks 公司从加拿大滑铁卢大学购得 MAPLE 软件的使用权，从而以 MAPLE 为“引擎”开发了符号数学工具箱（Symbolic Math Toolbox）。

MATLAB 软件包括五大通用功能：数值计算功能（Nemeric）；符号运算功能（Symbolic）；数据可视化功能（Graphic）；数据图形文字统一处理功能（Notebook）和建模仿真可视化功能（Simulink）。其中，符号运算功能的实现是通过请求 MAPLE 内核计算并将结果返回到 MATLAB 命令窗口。该软件有三大特点：一是功能强大；二是界面友善、语言自然；三是开放性强。目前，Mathworks 公司已推出 30 多个应用工具箱。MATLAB 在线性代数、矩阵分析、数值及优化、数理统计和随机信号分析、电路与系统、系统动力学、信号和图像处理、控制理论分析和系统设计、过程控制、建模和仿真、通信系统、以及财政金融等众多领域的理论研究和工程设计中得到了广泛应用。

MATLAB 在信号与系统中的应用主要包括符号运算和数值计算仿真分析。由于信号与系统课程的许多内容都是基于公式演算，而 MATLAB 借助符号数学工具箱提供的符号运算功能能基本满足信号与系统课程的需求。例如，解微分方程、傅里叶正反变换、拉普拉斯正反变换、 z 正反变换等。MATLAB 在信号与系统中的另一主要应用是数值计算与仿真分析，主要包括函数波形绘制、函数运算、冲激响应与阶跃响应仿真分析、信号的时域分析、信号的频谱分析、系统的 S 域分析、零极点图绘制等内容。数值计算仿真分析可以帮助学生更深入理解信号与系统的理论知识，并为将来使用 MATLAB 进行信号处理领域的各种分析和实际应用打下基础。

1.2 MATLAB 软件使用入门

1.2.1 MATLAB 软件的环境介绍

MATLAB 6.5 的工作桌面由标题栏、菜单栏、工具栏、命令窗口（Command Window）、工作空间窗口（Workspace）、当前目录窗口（Current Directory）、历史命令窗口（Command History）及状态栏组成，从而为用户使用 MATLAB 提供了集成的交互式图形界面，如

图 1-1 所示。

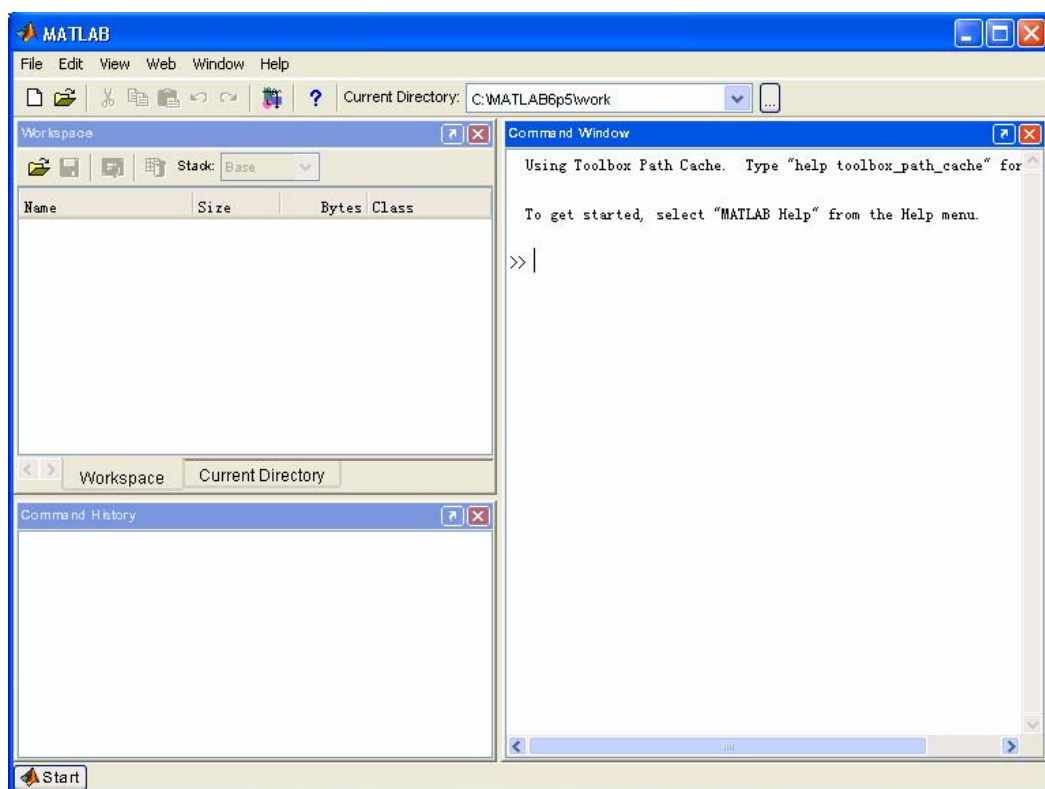


图 1-1 MATLAB 的工作桌面

MATLAB 的命令窗口是接收用户输入命令及输出数据显示的窗口，几乎所有的 MATLAB 行为都是在命令窗口进行的。当启动 MATLAB 软件时，命令窗口就做好了接收指令和输入的准备，并出现命令提示符（>>）。在命令提示符后输入指令通常会创建一个或多个变量。变量可以是多种类型的，包括函数和字符串，但通常的变量只是数据。这些变量被放置在 MATLAB 的工作空间中，工作空间窗口提供了变量的一些重要信息，包括变量的名称、维数大小、占用内存大小、以及数据类型等信息。查看工作空间的另一种方法是使用 `whos` 命令。在命令提示符后输入 `whos` 命令，工作空间中的内容概要将作为输出显示在命令窗口中。

有的命令可以用来清除不必要的数据，同时释放部分系统资源。`clear` 命令可以用来清除工作空间的所有变量，如果要清除某一特定变量则需要在 `clear` 命令后加上该变量的名称。另外，`clc` 命令用来清除命令窗口的内容。

如果希望将 MATLAB 所创建的变量及重要数据保留下来，则使用 `save` 命令，并在其后加上文件名，即可将整个工作空间保存为一个扩展名为 `.mat` 的文件。使用 `load` 命令，并在其后加上文件名，则可将 MATLAB 数据文件（`.mat` 文件）中的数据加载到工作空间中。MATLAB 历史命令窗口记录了每次输入的命令。在该窗口中可以对以前的历史命令进行查看、复制或者直接运行。

对于初学者而言，需要掌握的最重要且最有用的命令应为 `help` 命令。MATLAB 命令和函数有数千个，而且许多命令的功能非常强大，调用形式多样。要想了解一个命令或函数，只需在命令提示符后输入 `help`，并加上该命令或函数的名称，则 MATLAB 会给出其详细帮助信息。另外，MATLAB 还精心设计了演示程序系统（Demo），内容包括 MATLAB 的内部主要函数和各个工具箱（Toolbox）的使用。初学者可以方便地通过这些演示程序及其给出的程序源代码进行直观的感受和学习。用户可以通过两种途径打开演示程序系统。一是在命令窗口中输入 `demo` 或 `demos` 命令并按 Enter 键；二是选择“help”→“Demos”菜单命令。

1.2.2 MATLAB 软件基本运算入门

1. MATLAB 软件的数值计算

（1）算术运算

MATLAB 可以像一个简单的计算器一样使用，不论是实数运算还是复数运算都能轻松完成。标量的加法、减法、除法和幂运算均可通过常规符号“+”、“-”、“*”、“/”、以及“^”来完成。对于复数中的虚数单位，MATLAB 采用预定义变量 `i` 或 `j` 表示，即 $i=j=\sqrt{-1}$ 。因此，一个复常量可用直角坐标形式来表示，例如，

```
>>A=-3-i*4
A=
-3.0000 - 4.0000i
```

将复常量 `-3-i4` 赋予了变量 `A`。

一个复常量还可用极坐标的形式来表示，例如，

```
>>B=2*exp(i*pi/6)
B=
1.7321 + 1.0000i
```

其中，`pi` 是 MATLAB 预定义变量， $pi=\pi$ 。

复数的实部和虚部可以通过 `real` 和 `imag` 运算符来实现，而复数的模和辐角可以通过 `abs` 和 `angle` 运算符来实现。但应注意辐角的单位为弧度。例如，复数 `A` 的模和辐角、复数 `B` 的实部和虚部的计算分别为

```
>>A_mag=abs(A)
A_mag=
5
>>A_rad=angle(A)
A_rad=
-2.2143
>>B_real=real(B)
```

```

B_real=
    1.7321
>>B_imag=imag(B)
B_imag=
    1.0000

```

如果将弧度值用“度”来表示，则可进行转换，即

```

>>A_deg=angle(A)*180/pi
A_deg=
   -126.8699

```

复数 A 的模可表示为 $|A| = \sqrt{AA^*}$ ，因此，其共轭复数可通过 `conj` 命令来实现，例如，

```

>>A_mag=sqrt(A*conj(A))
A_mag=
    5

```

(2) 向量运算

向量是组成矩阵的基本元素之一，MATLAB 具有关于向量运算的强大功能。一般地，向量被分为行向量和列向量。生成向量的方法有很多，我们主要介绍两种。

①直接输入向量：即把向量中的每个元素列举出来。向量元素要用“[]”括起来，元素之间可用空格、逗号分隔生成行向量，用分号分隔生成列向量。例如，

```

>>A=[1,3,5,21]
A=
    1     3     5    21
>>B=[1;3;5;21]
B=
    1
    3
    5
   21

```

②利用冒号表达式生成向量：这种方法用于生成等步长或均匀等分的行向量，其表达式为 $x=x0:step:xn$ 。其中， $x0$ 为初始值； $step$ 表示步长或增量； xn 为结束值。如果 $step$ 值缺省，则步长默认为 1。例如，

```

>>C=0:2:10
C=
    0     2     4     6     8    10
>>D=0:10
D=
    0     1     2     3     4     5     6     7     8     9    10

```

在连续时间信号和离散时间信号的表示过程中，我们经常要用到冒号表达式。例如，对于 $0 \leq t \leq 1$ 范围内的连续信号，可用冒号表达式 “ $t=0:0.001:1$;” 来近似表达该区间，此时，向量 t 表示该区间以 0.001 为间隔的 1001 个点。

如果一个向量或一个标量与一个数进行运算，即 “+”、“-”、“*”、“/”、以及 “^” 运算，则运算结果是将该向量的每一个元素与这个数逐一进行相应的运算所得到的新的向量。例如，

```
>>C=0:2:10;
>>E=C/4
E=
    0    0.5000    1.0000    1.5000    2.0000    2.5000
```

其中，第一行语句结束的分号是为了不显示 C 的结果；第二句没有分号则显示出 E 的结果。

一个向量中元素的个数可以通过命令 “length” 获得，例如，

```
>>t=0:0.001:1;
>>L=length(t)
L=
    1001
```

(3) 矩阵运算

MATLAB 又称矩阵实验室，因此，MATLAB 中矩阵的表示十分方便。例如，输入

矩阵 $\begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{bmatrix}$ 在 MATLAB 命令窗口中可输入下列命令得到，即

```
>>a=[11 12 13;21 22 23;31 32 33]
a=
    11    12    13
    21    22    23
    31    32    33
```

其中，命令中整个矩阵用括号 “[]” 括起来；矩阵每一行的各个元素必须用逗号 “,” 或空格分开；矩阵的不同行之间必须用分号 “;” 或者按 Enter 键分开。

在矩阵的加减运算中，矩阵维数相同才能实行加减运算。矩阵的加法或减法运算将矩阵的对应元素分别进行加法或减法运算。在矩阵的乘法运算中，要求两矩阵必须维数相容，即第一个矩阵的列数必须等于第二个矩阵的行数。例如，

```
>>a=[1 2 3;4 5 6]
a=
    1    2    3
    4    5    6
>>b=[1 2; 3 4;5 6]
```

```

b=
     1     2
     3     4
     5     6

>>c=a*b

c=
    22    28
    49    64

```

MATLAB 中矩阵的点运算指维数相同的矩阵位置对应元素进行的算术运算，标量常数可以和矩阵进行任何点运算。常用的点运算包括“.*”、“./”、“.\”、“.^”等。矩阵的加法和减法是在对应元素之间进行的，所以不存在点加法或点减法。

点乘运算，又称 Hadamard 乘积，是指两维数相同的矩阵或向量对应元素相乘，表示为 $C=A.*B$ 。点除运算是指两维数相同的矩阵或向量中各元素独立的除运算，包括点右除和点左除。其中，点右除表示为 $C=A./B$ ，意思是 A 对应元素除以 B 对应元素；点左除表示为 $C=A.\backslash B$ ，意思是 B 对应元素除以 A 对应元素。点幂运算指两维数相同的矩阵或向量各元素独立的幂运算，表达式为 $C=A.^B$ 。

【实例 1-1】 已知 t 为一向量，用 MATLAB 命令计算 $y = \frac{\sin(t)e^{-2t} + 5}{\cos(t) + t^2 + 1}$ 在 $0 \leq t \leq 1$ 区

间上对应的值。

解：表达式中的运算都是点运算，MATLAB 源程序为

```

>>t=0:0.01:1;
>>y=(sin(t).*exp(-2*t)+5)./(cos(t)+t.^2+1);
>>plot(t,y),xlabel('t'),ylabel('y')

```

这里，我们未将 y 向量的结果显示出来，而是利用 plot 命令将结果绘出图形来，如图 1-2 所示。

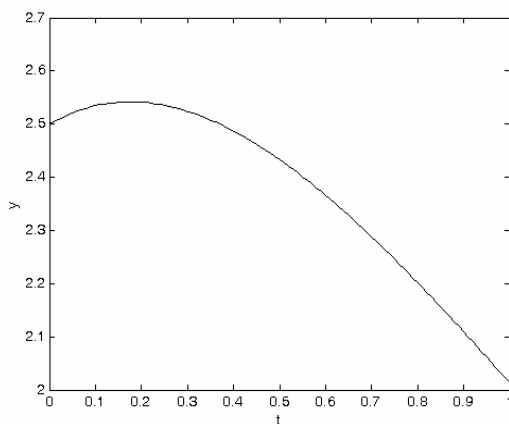


图 1-2 实例 1-1 的结果

2. MATLAB 软件的符号运算

MATLAB 符号运算工具箱提供的函数命令是专门研究符号运算功能的。符号运算是指符号之间的运算，其运算结果仍以标准的符号形式表达。符号运算是 MATLAB 的一个极其重要的组成部分，符号表示的解析式比数值解具有更好的通用性。在使用符号运算之前必须定义符号变量，并创建符号表达式。定义符号变量的语句格式为

```
syms 变量名
```

其中，各个变量名须用空格隔开。例如，定义 x、y、z 三个符号变量的语句格式为

```
>>syms x y z
```

我们可以用 whos 命令来查看所定义的符号变量，即

```
>>clear
```

```
>>syms x y z
```

```
>>whos
```

Name	Size	Bytes	Class
x	1x1	126	sym object
y	1x1	126	sym object
z	1x1	126	sym object

Grand total is 6 elements using 378 bytes

可见，变量 x、y、z 必须通过符号对象定义，即 sym object，才能参与符号运算。

另一种定义符号变量的语句格式为

```
sym('变量名')
```

例如，x、y、z 三个符号变量定义的语句格式为

```
>>x=sym('x');
```

```
>>y=sym('y');
```

```
>>z=sym('z');
```

sym 语句还可以用来定义符号表达式，语句格式为

```
sym('表达式')
```

例如，定义表达式 $x+1$ 为符号表达式对象，语句为

```
>>sym('x+1');
```

另一种创建符号表达式的方法是先定义符号变量，然后直接写出符号表达式。例如，

在 MATLAB 中创建符号表达式 $y = \frac{\sin(t)e^{-2t} + 5}{\cos(t) + t^2 + 1}$ ，其 MATLAB 源程序为

```
>>syms t
```

```
>>y=(sin(t).*exp(-2*t)+5)./(cos(t)+t.^2+1)
```

```
y=
```

```
(sin(t)*exp(-2*t)+5)/(cos(t)+t^2+1)
```

例如，符号算术运算的 MATLAB 源程序为

```
>>clear
>>syms a b
>>f1=1/(a+1);
>>f2=2*a/(a+b);
>>f3=(a+1)*(b-1)*(a-b);
>>f1+f2
ans=
      1/(a+1)+2*a/(a+b)
>>f1*f3
ans=
      (b-1)*(a-b)
>>f1/f3
ans=
      1/(a+1)^2/(b-1)/(a-b)
```

在符号运算中，可以用“simple”或者“simplify”函数来化简运算结果，例如，

```
>>syms x
>>f1=sin(x)^2;
>>f2=cos(x)^2;
>>y=f1+f2
y=
      sin(x)^2+cos(x)^2
>>y=simplify(y)
y=
      1
```

1.2.3 MATLAB 软件简单二维图形绘制

MATLAB 的 plot 命令是绘制二维曲线的基本函数，它为数据的可视化提供了方便的途径。例如，函数 $y = f(x)$ 关于变量 x 的曲线绘制的语句格式为

```
>>plot(x,y)
```

其中，输出以向量 x 为横坐标，向量 y 为纵坐标，且按照向量 x 、 y 中元素的排列顺序有序绘制图形。但向量 x 与 y 必须拥有相同的长度。

绘制多幅图形的语句格式为

```
>>plot(x1,y1,'str1',x2,y2,'str2',...)
```

其中，用 str1 制定的方式，输出以 x_1 为横坐标、 y_1 为纵坐标的图形。用 str2 制定的方式，输出以 x_2 为横坐标、 y_2 为纵坐标的图形。若省略 str，则 MATLAB 自动为每条曲

线选择颜色与线型。

图形完成后，可以通过几个命令来调整显示结果。如 `grid on` 用来显示格线；`axis([xmin,xmax,ymin,ymax])` 函数调整坐标轴的显示范围。其中，括号内的“,”可用空格代替；`xlabel` 和 `ylabel` 命令可为横坐标和纵坐标加标注，标注的字符串必须用单引号引起来；`title` 命令可在图形顶部加注标题。

【实例 1-2】 用 MATLAB 命令绘制函数 $y = \sin(10\pi t) + \frac{1}{\cos(\pi t) + 2}$ 的波形图。

解：MATLAB 源程序为

```
>>t=0:0.01:5;
>>y=sin(5*pi*t)+1./(cos(pi*t)+2);
>>plot(t,y)
>>axis([0,5,-1 2,5])
>>xlabel('t'),ylabel('y'),
>>grid on
```

程序运行结果如图 1-3 所示。

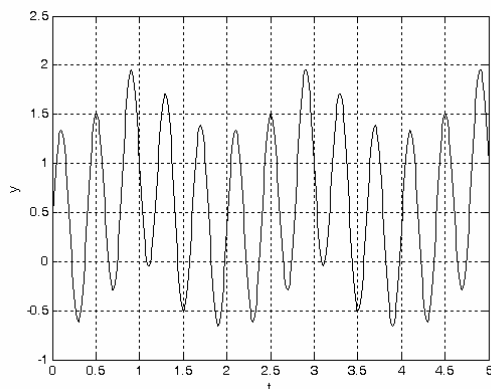


图 1-3 实例 1-2 的函数波形图

用 `subplot` 命令可在一个图形窗口中按照规定的排列方式同时显示多个图形，方便图形的比较。其语句格式为

```
>>subplot(m,n,p)
```

或者

```
>>subplot(mnp)
```

其中，`m` 和 `n` 表示在一个图形窗口中显示 `m` 行 `n` 列个图像，`p` 表示第 `p` 个图像区域，即在第 `p` 个区域作图。例如，比较正弦信号相位差的 MATLAB 源程序为

```
>>t=0:0.01:3;
>>y1=sin(2*pi*t);
>>y2=sin(2*pi*t+pi/6);
>>subplot(211),plot(t,y1)
>>xlabel('t'),ylabel('y1'),title('y1=sin(2*pi*t)')
```

```
>>subplot(212),plot(t,y2)
>>xlabel('t'),ylabel('y2'),title('y2=sin(2*pi*t+pi/6)')
```

程序运行结果如图 1-4 所示。

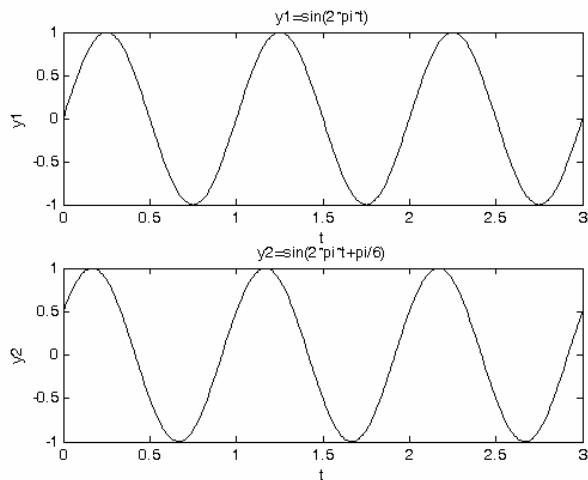


图 1-4 正弦信号相位差比较

除了 plot 命令外，MATLAB 提供了 ezplot 命令绘制符号表达式的曲线，其语句格式为

```
>>ezplot(y, [a,b])
```

其中，[a, b]参数表示符号表达式的自变量取值范围，默认值为 $[0, 2\pi]$ 。

【实例 1-3】 利用 MATLAB 的 ezplot 命令绘出函数 $y = -16x^2 + 64x + 96$ 的波形图。

解：MATLAB 源程序为

```
>>syms x
>>y='-16*x^2+64*x+96';
>>ezplot(y,[0,5])
>>xlabel('t'),ylabel('y'),
>>grid on
```

程序运行结果如图 1-5 所示。

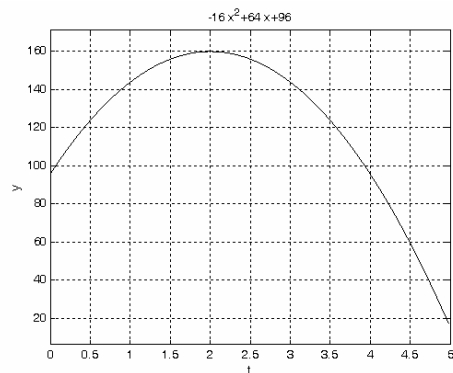


图 1-5 实例 1-3 的函数波形图

在绘图过程中，我们可利用“hold on”命令来保持当前图形，继续在当前图形状态下绘制其他图形，即可在同一窗口下绘制多幅图形。“hold off”命令用来释放当前图形窗口，绘制下一幅图形作为当前图形。

1.2.4 M 文件

MATLAB 是解释型语言，也就是说在 MATLAB 命令行中输入的命令在当前 MATLAB 进程中被解释运行，无需编译和链接等。MATLAB 文件分为两类：M 脚本文件（M-Script）和 M 函数（M-function），它们均为由 ASCII 码构成的文件，该文件可直接在文本编辑器中编写，称为 M 文件，保存的文件扩展名是.m。

M 脚本文件包含一族由 MATLAB 语言所支持的语句，并保存为 M 文件。它类似于 DOS 下的批处理文件，不需要在其中输入参数，也不需要给出输出变量来接受处理结果。脚本仅是若干命令或函数的集合，用于执行特定的功能。其执行方式很简单，用户只需在 MATLAB 的提示符>>下键入该 M 文件的文件名，这样 MATLAB 就会自动执行该 M 文件中的各条语句，并将结果直接返回到 MATLAB 工作空间中。脚本 M 文件实际上是一系列 MATLAB 命令的集合，它的作用与在 MATLAB 命令窗口输入的一系列命令等效。

M 函数文件不同于 M 脚本文件，是一种封装结构，通过外界提供输入量而得到函数文件的输出结果。函数是接受入口参数返回出口参数的 M 文件，程序在自己的工作空间中操作变量，与工作空间分开，无法访问。M 函数文件和 M 脚本文件都是在编辑器中生成，通常以关键字 function 引导“函数声明行”，并罗列出函数与外界联系的全部“标称”输入输出宗量。它的一般形式为

```
function [output 1, output 2,...] = functionname(input1, input2,...)
%[output 1, output 2,...] = functionname(input1, input2,...) Functionname
%Some comments that explain what the function does go here.
MATLAB command 1;
MATLAB command 2;
MATLAB command 3;
.....
```

该函数的 M 文件名是 functionname.m，在 MATLAB 命令窗口中可被其他 M 文件调用，例如，

```
>> [output1, output2] = functionname(input1, input2)
```

注意，MATLAB 忽略了“%”后面的所有文字，因此，可以利用该符号写注释。以“;”结束一行可以停止输出打印，在一行的最后输入“...”可以续行，以便在下一行继续输入指令。M 函数格式是 MATLAB 程序设计的主流，在一般情况下，不建议使用 M 脚本文件格式编程。

1.2.5 MATLAB 程序流程控制

MATLAB 与其他高级编程语言一样，是一种结构化的编程语言。MATLAB 程序流程控制结构一般可分为顺序结构、循环结构、以及条件分支结构。MATLAB 中实现顺序结构的方法非常简单，只需将程序语句按顺序排列即可。在 MATLAB 中，循环结构可以由 for 语句循环结构和 while 语句循环结构两种方式来实现。条件分支结构可以由 if 语句分支结构和 switch 语句分支结构两种方式来实现。下面我们主要介绍这几种程序流程控制。

1. for 循环结构

for 循环结构用于在一定条件下多次循环执行处理某段指令，其语法格式为

```
for  循环变量=初值：增量：终值
    循环体
end
```

循环变量一般被定义为一个向量，这样循环变量从初值开始，循环体中的语句每被执行一次，变量值就增加一个增量，直到变量等于终值为止。增量可以根据需要设定，默认时为 1。end 代表循环体的结束部分。

例如，用 for 循环结构求 $1+2+3+\cdots+100$ 的和，其 MATLAB 源程序为

```
>>sum=0;
>>for i=1:100
    sum=sum+i;
end
>>sumsum=
    5050
```

2. while 循环结构

while 循环结构也用于循环执行处理某段指令，但是与 for 循环结构不同的是在执行循环体之前先要判断循环执行的条件是否成立，即逻辑表达式为“真”还是“假”，如果条件成立，则执行；如果条件不成立，则终止循环。其语法格式为

```
while 逻辑表达式
    循环体
end
```

例如，用 while 循环结构求 $1+2+3+\cdots+100$ 的和，其 MATLAB 源程序为

```
>>sum=0;i=0;
>>while i<100
```

```

i=i+1;
sum=sum+i;
end
>>sum
sum=
    5050

```

从上述 MATLAB 源程序可以看出，while 循环结构是通过判断逻辑表达式 $i < 100$ 是否为“真”，而决定是否执行循环体。

3. if 分支结构

if 条件分支结构是通过判断逻辑表达式是否成立来决定是否执行制定的程序模块。其语法格式有两种，一种是单分支结构；另一种为多分支结构。其中，单分支结构语法格式为

```

if    逻辑表达式
    程序模块
end

```

单分支结构语法格式的含义是，如果逻辑表达式为“真”，则执行程序模块，否则跳过该分支结构，按顺序结构执行下面的程序。

多分支结构的语法格式为

```

if    逻辑表达式 1
    程序模块 1
else if    逻辑表达式 2    （可选）
    程序模块 2
.....
else
    程序模块 n
end

```

多分支结构语法格式可理解为：首先判断 if 条件分支结构中的逻辑表达式 1 是否成立，如果成立则执行程序模块 1；否则继续判断 else if 条件分支结构中的逻辑表达式 2，如果成立则执行程序模块 2；依次下去，如果结构中所有条件都不成立，则执行程序模块 n。

例如，用 if 条件分支结构可实现百分制考试分数分级，其 MATLAB 源程序为

```

>>s=input('输入 score= '); %屏幕提示输入 x=,由键盘输入值赋给 x
>>if s>=90
    rank='A'
elseif s>=80

```

```

        rank='B'
elseif s>=70
        rank='C'
elseif s>=60
        rank='D'
else
        rank='E'
end

```

4. switch 分支结构

switch 分支结构是根据表达式的取值结果不同来选择执行的程序模块，其语法格式为

```

switch 表达式
    case 常量 1
        程序模块 1
    case 常量 2
        程序模块 2
    ....
    otherwise
        程序模块 n
end

```

其中，switch 后面的表达式可以是任何类型，如数字、字符串等。当表达式的值与 case 后面的常量相等时，就执行对应的程序模块；如果所有常量都与表达式的值不等时，则执行 otherwise 后面的程序模块。

例如，用 switch 分支结构也可实现百分制考试分数分级，其 MATLAB 源程序为

```

>>s=input('输入 score= ');
>>switch fix(s/10) %利用 fix 函数舍去小数部分取最近整数
    case {10,9}
        rank='A'
    case 8
        rank='B'
    case 7
        rank='C'
    case 6
        rank='D'
    otherwise

```

```
rank='E'  
end
```

除了上述介绍的几种程序流程控制结构外，MATLAB 为实现交互控制程序流程还提供了 `continue`、`break`、`pause`、`input`、`error`、`disp` 等命令。读者可通过 `doc` 或者 `help` 命令查看它们的具体使用。

第 2 章 离散时间信号

2.1 实验原理及实例分析

2.1.1 离散时间信号在 MATLAB 中的表示

离散时间信号是指在离散时刻才有定义的信号，简称离散信号，或者序列。离散序列通常用 $x(n)$ 来表示，自变量必须是整数。

离散时间信号的波形绘制在 MATLAB 中一般用 `stem` 函数。`stem` 函数的基本用法和 `plot` 函数一样，它绘制的波形图的每个样本点上有一个小圆圈，默认是空心的。如果要实心，需使用参数“`fill`”、“`filled`”，或者参数“`.`”。由于 MATLAB 中矩阵元素的个数有限，所以 MATLAB 只能表示一定时间范围内有限长度的序列；而对于无限序列，也只能在一定时间范围内表示出来。类似于连续时间信号，离散时间信号也有一些典型的离散时间信号。

1. 单位取样序列

单位取样序列 $\delta(n)$ ，也称为单位冲激序列，定义为

$$\delta(n) = \begin{cases} 1 & (n = 0) \\ 0 & (n \neq 0) \end{cases} \quad (12-1)$$

要注意，单位冲激序列不是单位冲激函数的简单离散抽样，它在 $n=0$ 处是取确定的值 1。在 MATLAB 中，冲激序列可以通过编写以下的 `impDT.m` 文件来实现，即

```
function y=impDT(n)
```

```
y=(n==0); %当参数为 0 时冲激为 1,否则为 0
```

调用该函数时 n 必须为整数或整数向量。

【实例 2-1】 利用 MATLAB 的 `impDT` 函数绘出单位冲激序列的波形图。

解： MATLAB 源程序为

```
>>n=-3:3;
>>x=impDT(n);
>>stem(n,x,'fill'),xlabel('n'),grid on
>>title('单位冲激序列')
>>axis([-3 3 -0.1 1.1])
```

程序运行结果如图 12-1 所示。

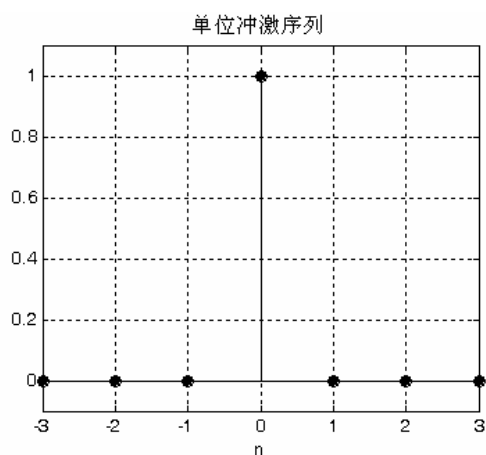


图 2-1 单位冲激序列

2. 单位阶跃序列

单位阶跃序列 $u(n)$ 定义为

$$u(n) = \begin{cases} 1 & (n \geq 0) \\ 0 & (n < 0) \end{cases} \quad (12-2)$$

在 MATLAB 中，冲激序列可以通过编写 uDT.m 文件来实现，即

```
function y=uDT(n)
y=n>=0;           %当参数为非负时输出 1
```

调用该函数时 n 也同样必须为整数或整数向量。

【实例 2-2】 利用 MATLAB 的 uDT 函数绘出单位阶跃序列的波形图。

解：MATLAB 源程序为

```
>>n=-3:5;
>>x=uDT(n);
>>stem(n,x,'fill'),xlabel('n'),grid on
>>title('单位阶跃序列')
>>axis([-3 5 -0.1 1.1])
```

程序运行结果如图 12-2 所示。

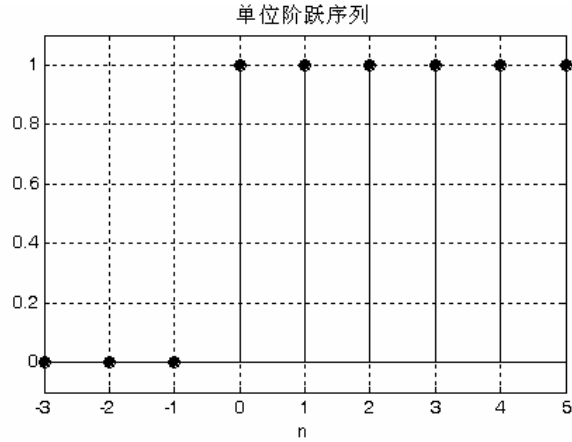


图 2-2 单位阶跃序列

3. 矩形序列

矩形序列 $R_N(n)$ 定义为

$$R_N(n) = \begin{cases} 1 & (0 \leq n \leq N-1) \\ 0 & (n < 0, n \geq N) \end{cases} \quad (12-3)$$

矩形序列有一个重要的参数，就是序列宽度 N 。 $R_N(n)$ 与 $u(n)$ 之间的关系为

$$R_N(n) = u(n) - u(n - N)$$

因此，用 MATLAB 表示矩形序列可利用上面所讲的 uDT 函数。

【实例 2-3】 利用 MATLAB 命令绘出矩形序列 $R_5(n)$ 的波形图。

解： MATLAB 源程序为

```
>>n=-3:8;
>>x=uDT(n)-uDT(n-5);
>>stem(n,x,'fill'),xlabel('n'),grid on
>>title('矩形序列')
>>axis([-3 8 -0.1 1.1])
```

程序运行结果如图 2-3 所示。

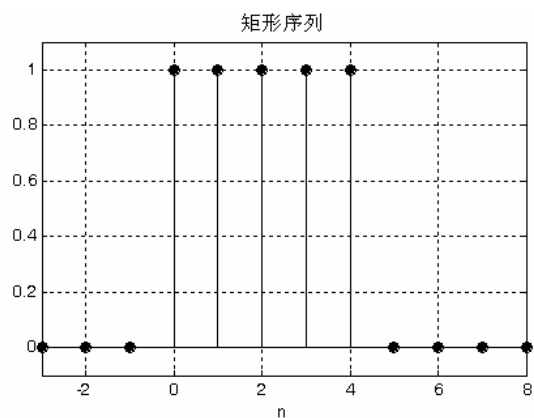


图 2-3 矩形序列

4. 单边指数序列

单边指数序列定义为

$$x(n) = a^n u(n) \quad (12-4)$$

【实例 2-4】 试用 MATLAB 命令分别绘制单边指数序列 $x_1(n) = 1.2^n u(n)$ 、

$x_2(n) = (-1.2)^n u(n)$ 、 $x_3(n) = (0.8)^n u(n)$ 、 $x_4(n) = (-0.8)^n u(n)$ 的波形图。

解：MATLAB 源程序为

```
>>n=0:10;
>>a1=1.2;a2=-1.2;a3=0.8;a4=-0.8;
>>x1=a1.^n;x2=a2.^n;x3=a3.^n;x4=a4.^n;
>>subplot(221)
>>stem(n,x1,'fill'),grid on
>>xlabel('n'),title('x(n)=1.2^{n}')
>>subplot(222)
>>stem(n,x2,'fill'),grid on
>>xlabel('n'),title('x(n)=(-1.2)^{n}')
>>subplot(223)
>>stem(n,x3,'fill'),grid on
>>xlabel('n'),title('x(n)=0.8^{n}')
>>subplot(224)
>>stem(n,x4,'fill'),grid on
>>xlabel('n'),title('x(n)=(-0.8)^{n}')
```

单边指数序列 n 的取值范围为 $n \geq 0$ 。程序运行结果如图 12-4 所示。从图可知，当 $|a| > 1$ 时，单边指数序列发散；当 $|a| < 1$ 时，该序列收敛。当 $a > 0$ 时，该序列均取正值；当 $a < 0$ 时，序列在正负摆动。

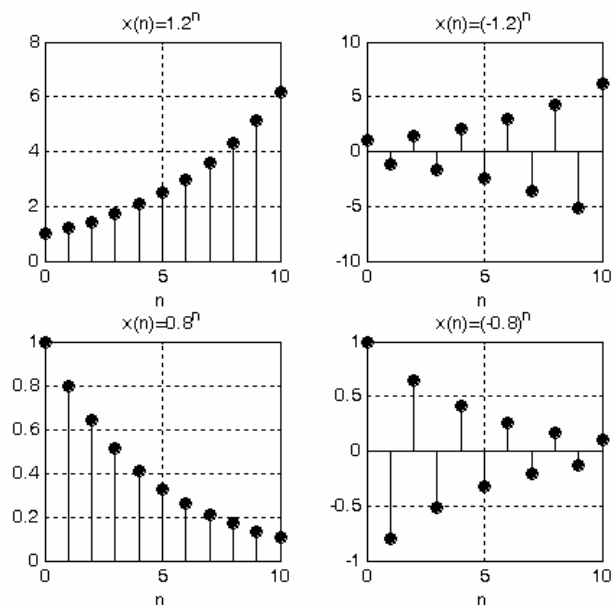


图 2-4 单边指数序列

5. 正弦序列

正弦序列定义为

$$x(n) = \sin(n\omega_0 + \varphi) \quad (12-5)$$

其中， ω_0 是正弦序列的数字域频率； φ 为初相。与连续的正弦信号不同，正弦序列的自变量 n 必须为整数。可以证明，只有当 $\frac{2\pi}{\omega_0}$ 为有理数时，正弦序列具有周期性。

【实例 2-5】 试用 MATLAB 命令绘制正弦序列 $x(n) = \sin(\frac{n\pi}{6})$ 的波形图。

解： MATLAB 源程序为

```
>>n=0:39;
>>x=sin(pi/6*n);
>>stem(n,x,'fill'),xlabel('n'),grid on
>>title('正弦序列')
>>axis([0,40,-1.5,1.5]);
```

程序运行结果如图 2-5 所示。

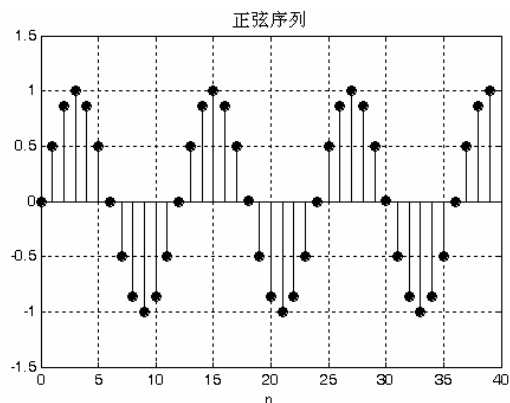


图 2-5 正弦序列

6. 复指数序列

复指数序列定义为

$$x(n) = e^{(a+j\omega_0)n} \quad (2-6)$$

当 $a=0$ 时，得到虚指数序列 $x(n) = e^{j\omega_0 n}$ ，式中 ω_0 是正弦序列的数字域频率。由欧拉公式知，复指数序列可进一步表示为

$$x(n) = e^{(a+j\omega_0)n} = e^{an} e^{j\omega_0 n} = e^{an} [\cos(n\omega_0) + j \sin(n\omega_0)] \quad (2-7)$$

与连续复指数信号一样，我们将复指数序列实部和虚部的波形分开讨论，得出如下结论：

- (1) 当 $a > 0$ 时，复指数序列 $x(n)$ 的实部和虚部分别是按指数规律增长的正弦振荡序列；
- (2) 当 $a < 0$ 时，复指数序列 $x(n)$ 的实部和虚部分别是按指数规律衰减的正弦振荡序列；
- (3) 当 $a = 0$ 时，复指数序列 $x(n)$ 即为虚指数序列，其实部和虚部分别是等幅的正弦振荡序列。

【实例 2-6】 用 MATLAB 命令画出复指数序列 $x(n) = 2e^{(-\frac{1}{10} + j\frac{\pi}{6})n}$ 的实部、虚部、模及相角随时间变化的曲线，并观察其时域特性。

解： MATLAB 源程序为

```
>>n=0:30;
>>A=2;a=-1/10;b=pi/6;
>>x=A*exp((a+i*b)*n);
>>subplot(2,2,1)
```

```

>>stem(n,real(x),'fill'),grid on
>>title('实部'),axis([0,30,-2,2]),xlabel('n')
>>subplot(2,2,2)
>>stem(n,imag(x),'fill'),grid on
>>title('虚部'),axis([0,30,-2,2]),xlabel('n')
>>subplot(2,2,3)
>>stem(n,abs(x),'fill'),grid on
>>title('模'),axis([0,30,0,2]),xlabel('n')
>>subplot(2,2,4)
>>stem(n,angle(x),'fill'),grid on
>>title('相角'),axis([0,30,-4,4]),xlabel('n')

```

程序运行后，产生如图 2-6 所示的波形。

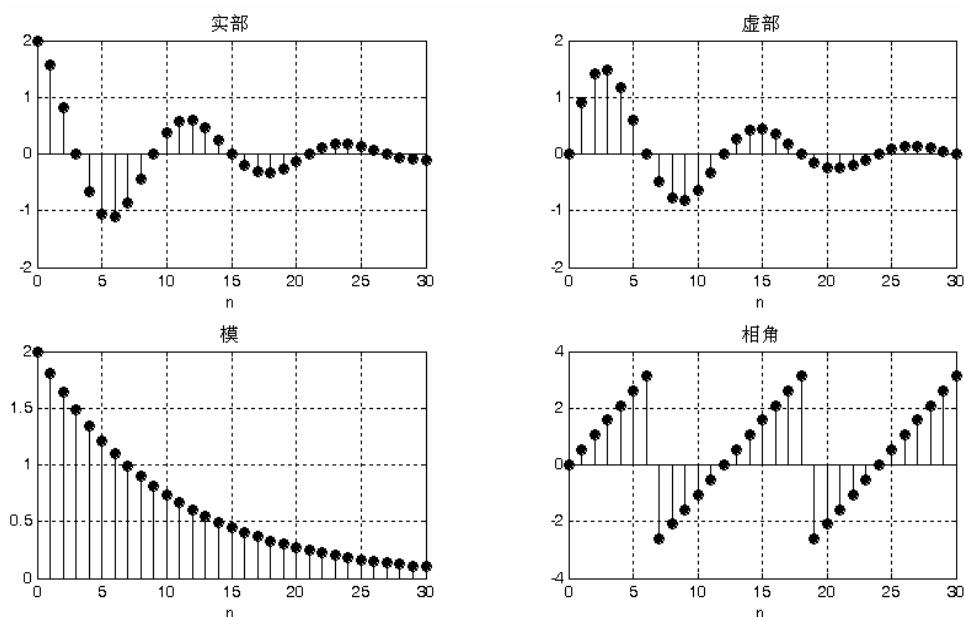


图 2-6 复指数序列

2.1.2 离散时间信号的基本运算

对离散时间序列实行基本运算可得到新的序列，这些基本运算主要包括加、减、乘、除、移位、反折等。两个序列的加减乘除是对应离散样点值的加减乘除，因此，可通过 MATLAB 的点乘和点除、序列移位和反折来实现，与连续时间信号处理方法基本一样。

【实例 2-7】 用 MATLAB 命令画出下列离散时间信号的波形图。

$$(1) \quad x_1(n) = a^n [u(n) - u(n - N)];$$

$$(2) \quad x_2(n) = x_1(n + 3)$$

$$(3) \quad x_3(n) = x_1(n-2);$$

$$(4) \quad x_4(n) = x_1(-n)$$

解：设 $a = 0.8$, $N = 8$, MATLAB源程序为

```
>>a=0.8;N=8;n=-12:12;
>>x=a.^n.*(uDT(n)-uDT(n-N));
>>n1=n;n2=n1-3;n3=n1+2;n4=-n1;
>>subplot(411)
>>stem(n1,x,'fill'),grid on
>>title('x1(n)'),axis([-15 15 0 1])
>>subplot(412)
>>stem(n2,x,'fill'),grid on
>>title('x2(n)'),axis([-15 15 0 1])
>>subplot(413)
>>stem(n3,x,'fill'),grid on
>>title('x3(n)'),axis([-15 15 0 1])
>>subplot(414)
>>stem(n4,x,'fill'),grid on
>>title('x4(n)'),axis([-15 15 0 1])
```

其波形如图2-7所示。

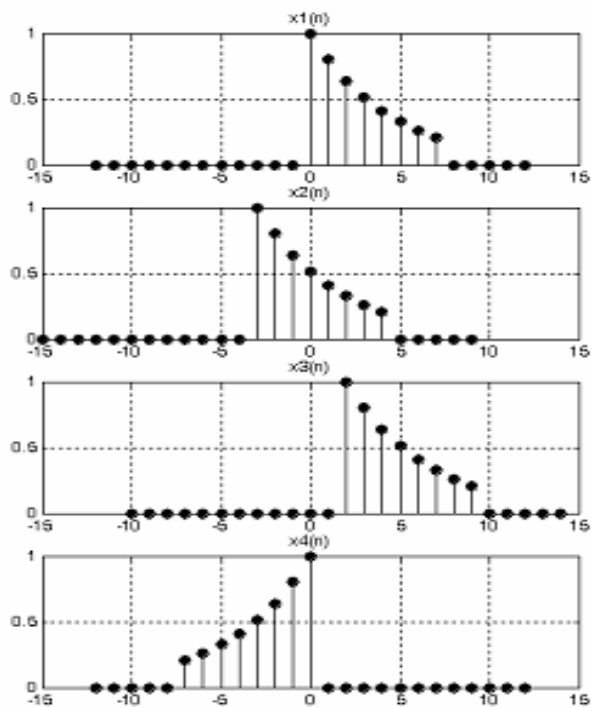


图 2-7 离散时间信号的基本运算及波形图

第 3 章 离散时间 LTI 系统的时域分析

3.1 实验原理及实例分析

3.1.1 离散时间系统的响应

离散时间 LTI 系统可用线性常系数差分方程来描述，即

$$\sum_{i=0}^N a_i y(n-i) = \sum_{j=0}^M b_j x(n-j) \quad (3-1)$$

其中， a_i ($i=0, 1, \dots, N$) 和 b_j ($j=0, 1, \dots, M$) 为实常数。

MATLAB 中函数 `filter` 可对式 (13-1) 的差分方程在指定时间范围内的输入序列所产生的响应进行求解。函数 `filter` 的语句格式为

$$y = \text{filter}(b, a, x)$$

其中， x 为输入的离散序列； y 为输出的离散序列； y 的长度与 x 的长度一样； b 与 a 分别为差分方程右端与左端的系数向量。

【实例 3-1】 已知某 LTI 系统的差分方程为

$$3y(n) - 4y(n-1) + 2y(n-2) = x(n) + 2x(n-1)$$

试用 MATLAB 命令绘出当激励信号为 $x(n) = (1/2)^n u(n)$ 时，该系统的零状态响应。

解：MATLAB 源程序为

```
>>a=[3 -4 2];  
>>b=[1 2];  
>>n=0:30;  
>>x=(1/2).^n;  
>>y=filter(b,a,x);  
>>stem(n,y,'fill'),grid on  
>>xlabel('n'),title('系统响应 y(n)')
```

程序运行结果如图 3-1 所示。

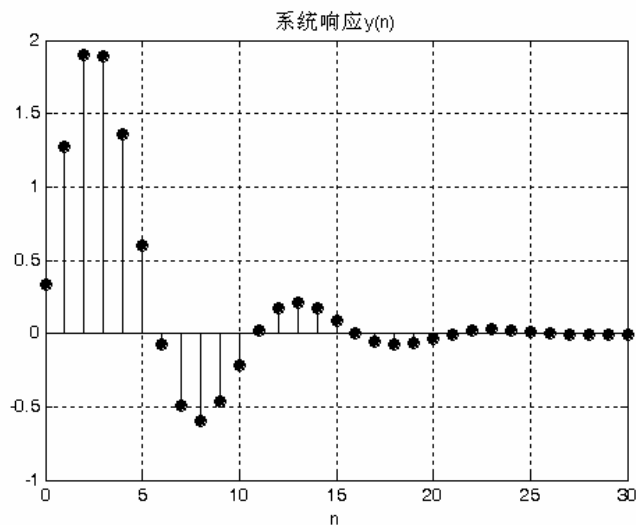


图 3-1 实例 3-1 系统的零状态响应

3.1.2 离散时间系统的单位取样响应

系统的单位取样响应定义为系统在 $\delta(n)$ 激励下系统的零状态响应，用 $h(n)$ 表示。MATLAB 求解单位取样响应可利用函数 `filter`，并将激励设为前面所定义的 `impDT` 函数。例如，求解实例 13-1 中系统的单位取样响应时，MATLAB 源程序为

```
>>a=[3 -4 2];
>>b=[1 2];
>>n=0:30;
>>x=impDT(n);
>>h=filter(b,a,x);
>>stem(n,h,'fill'),grid on
>>xlabel('n'),title('系统单位取样响应 h(n)')
```

程序运行结果如图 3-2 所示。

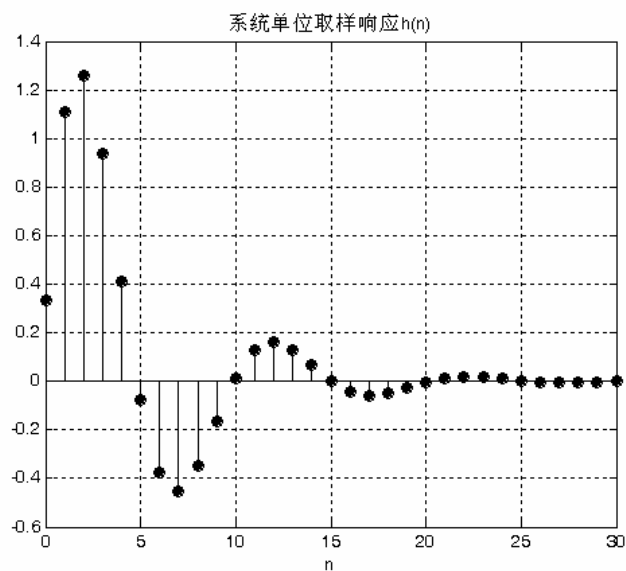


图 3-2 实例 13-1 的系统单位取样响应

MATLAB 另一种求单位取样响应的方法是利用控制系统工具箱提供的函数 `impz` 来实现。`impz` 函数的常用语句格式为

$$\text{impz}(b,a,N)$$

其中，参数 N 通常为正整数，代表计算单位取样响应的样值个数。

【实例 3-2】 已知某 LTI 系统的差分方程为

$$3y(n) - 4y(n-1) + 2y(n-2) = x(n) + 2x(n-1)$$

利用 MATLAB 的 `impz` 函数绘出该系统的单位取样响应。

解：MATLAB 源程序为

```
>>a=[3 -4 2];
>>b=[1 2];
>>n=0:30;
>>impz(b,a,30),grid on
>>title('系统单位取样响应 h(n)')
```

程序运行结果如图 3-3 所示，比较图 3-2 和图 3-3，不难发现结果相同。

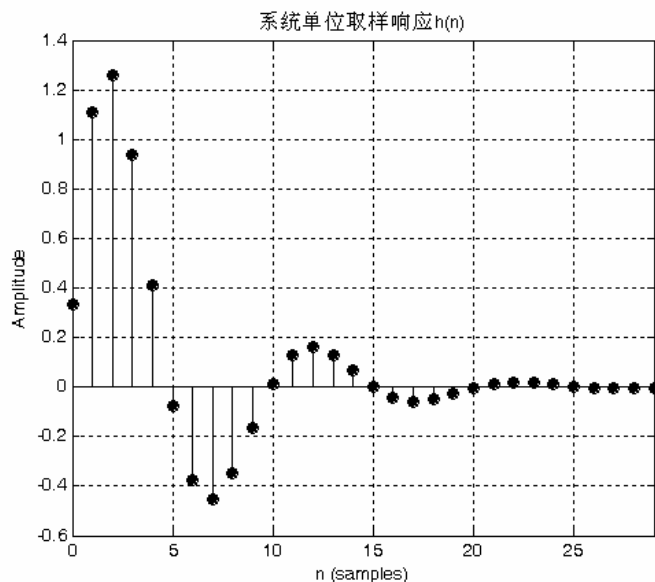


图 3-3 系统单位取样响应

3.1.3 离散时间信号的卷积和运算

由于系统的零状态响应是激励与系统的单位取样响应的卷积，因此卷积运算在离散时间信号处理领域被广泛应用。离散时间信号的卷积定义为

$$y(n) = x(n) * h(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) \quad (3-2)$$

可见，离散时间信号的卷积运算是求和运算，因而常称为“卷积和”。

MATLAB 求离散时间信号卷积和的命令为 `conv`，其语句格式为

$$y = \text{conv}(x, h)$$

其中， x 与 h 表示离散时间信号值的向量； y 为卷积结果。用 MATLAB 进行卷积和运算时，无法实现无限的累加，只能计算时限信号的卷积。

例如，利用 MATLAB 的 `conv` 命令求两个长为 4 的矩形序列的卷积和，即 $g(n) = [u(n) - u(n-4)] * [u(n) - u(n-4)]$ ，其结果应是长为 7 ($4+4-1=7$) 的三角序列。用向量 `[1 1 1 1]` 表示矩形序列，MATLAB 源程序为

```
>>x1=[1 1 1 1];
>>x2=[1 1 1 1];
>>g=conv(x1,x2)
g=
    1     2     3     4     3     2     1
```

如果要绘出图形来，则利用 `stem` 命令，即

```
>>n=1:7;
```

```
>>stem(n,g,'fill'),grid on,xlabel('n')
```

程序运行结果如图 13-4 所示。

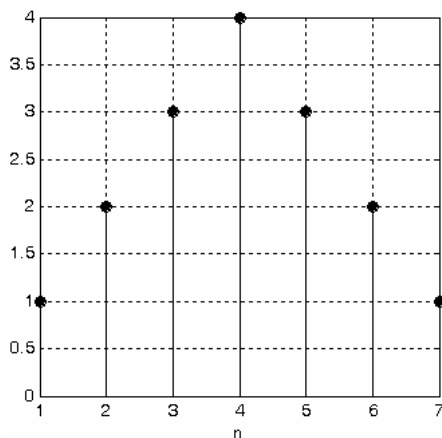


图 3-4 卷积结果图

对于给定函数的卷积和，我们应计算卷积结果的起始点及其长度。两个时限序列的卷积和长度一般等于两个序列长度的和减 1。

【实例 3-3】 已知某系统的单位取样响应为 $h(n) = 0.8^n[u(n) - u(n-8)]$ ，试用 MATLAB 求当激励信号为 $x(n) = u(n) - u(n-4)$ 时，系统的零状态响应。

解：MATLAB 中可通过卷积求解零状态响应，即 $x(n) * h(n)$ 。由题意可知，描述 $h(n)$ 向量的长度至少为 8，描述 $x(n)$ 向量的长度至少为 4，因此为了图形完整美观，我们将 $h(n)$ 向量和 $x(n)$ 向量加上一些附加的零值。MATLAB 源程序为

```
>>nx=-1:5;           %x(n)向量显示范围(添加了附加的零值)
>>nh=-2:10;          %h(n)向量显示范围(添加了附加的零值)
>>x=uDT(nx)-uDT(nx-4);
>>h=0.8.^nh.*(uDT(nh)-uDT(nh-8));
>>y=conv(x,h);
>>ny1=nx(1)+nh(1);    %卷积结果起始点
>>%卷积结果长度为两序列长度之和减 1,即 0 到(length(nx)+length(nh)-2)
>>%因此卷积结果的时间范围是将上述长度加上起始点的偏移值
>>ny=ny1+(0:(length(nx)+length(nh)-2));
>>subplot(311)
>>stem(nx,x,'fill'),grid on
>>xlabel('n'),title('x(n)')
>>axis([-4 16 0 3])
>>subplot(312)
>>stem(nh,h,'fill'),grid on
```

```

>>xlabel('n'),title('h(n)')
>>axis([-4 16 0 3])
>>subplot(313)
>>stem(ny,y,'fill'),grid on
>>xlabel('n'),title('y(n)=x(n)*h(n)')
>>axis([-4 16 0 3])

```

程序运行结果如图 3-5 所示。

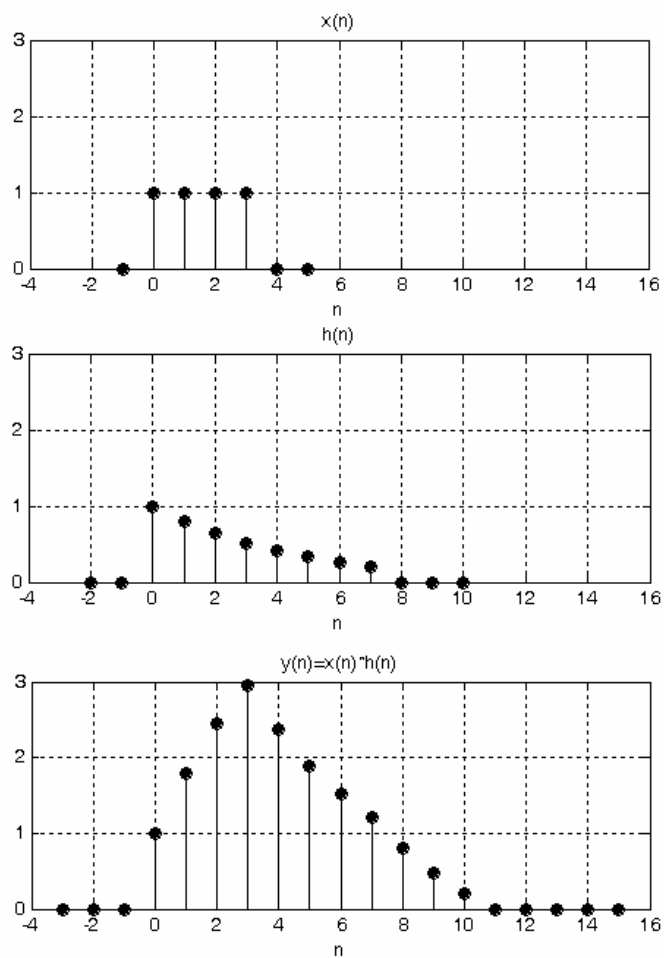


图 3-5 利用卷积和法求解系统的零状态响应

第 4 章 z 变换及离散时间 LTI 系统的 z 域分析

4.1 实验原理及实例分析

4.1.1 z 正反变换

序列 $x(n)$ 的 z 变换定义为

$$X(z) = Z[x(n)] = \sum_{n=-\infty}^{\infty} x(n)z^{-n} \quad (4-1)$$

其中，符号 Z 表示取 z 变换， z 是复变量。相应地，单边 z 变换定义为

$$X(z) = Z[x(n)] = \sum_{n=0}^{\infty} x(n)z^{-n} \quad (4-2)$$

MATLAB 符号数学工具箱提供了计算离散时间信号单边 z 变换的函数 `ztrans` 和 z 反变换函数 `iztrans`，其语句格式分别为

$$Z = \text{ztrans}(x)$$

$$x = \text{iztrans}(z)$$

上式中的 x 和 Z 分别为时域表达式和 z 域表达式的符号表示，可通过 `sym` 函数来定义。

【实例 4-1】 试用 `ztrans` 函数求下列函数的 z 变换。

$$(1) \quad x(n) = a^n \cos(\pi n) u(n); \quad (2) \quad x(n) = [2^{n-1} - (-2)^{n-1}] u(n)。$$

解：(1) z 变换 MATLAB 源程序为

```
>>x=sym('a^n*cos(pi*n)');
```

```
>>Z=ztrans(x);
```

```
>>simplify(Z)
```

```
ans=
```

$$z/(z+a)$$

(2) z 变换 MATLAB 源程序为

```
>>x=sym('2^(n-1)-(-2)^(n-1)');
```

```
>>Z=ztrans(x);
```

```
>>simplify(Z)
```

```
ans=
```

$$z^2/(z-2)/(z+2)$$

【实例 4-2】 试用 iztrans 函数求下列函数的 z 反变换。

$$(1) \quad X(z) = \frac{8z - 19}{z^2 - 5z + 6} \quad (2) \quad X(z) = \frac{z(2z^2 - 11z + 12)}{(z-1)(z-2)^3}$$

解：(1) z 反变换 MATLAB 源程序为

```
>>Z=sym('(8*z-19)/(z^2-5*z+6)');
>>x=iztrans(Z);
>>simplify(x)
ans=
-19/6*charfcn[0](n)+5*3^(n-1)+3*2^(n-1)
```

其中，charfcn[0](n)是 $\delta(n)$ 函数在 MATLAB 符号工具箱中的表示，反变换后的函数形式为

$$x(n) = -\frac{19}{6}\delta(n) + (5 \times 3^{n-1} + 3 \times 2^{n-1})u(n)。$$

(2) z 反变换 MATLAB 源程序为

```
>>Z=sym('z*(2*z^2-11*z+12)/(z-1)/(z-2)^3');
>>x=iztrans(Z);
>>simplify(x)
ans=
-3+3*2^n-1/4*2^n*n-1/4*2^n*n^2
```

其函数形式为 $x(n) = (-3 + 3 \times 2^n - \frac{1}{4}n2^n - \frac{1}{4}n^2 2^n)u(n)$ 。

如果信号的 z 域表示式 $X(z)$ 是有理函数，进行 z 反变换的另一个方法是对 $X(z)$ 进行部分分式展开，然后求各简单分式的 z 反变换。设 $X(z)$ 的有理分式表示为

$$X(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \cdots + b_m z^{-m}}{1 + a_1 z^{-1} + a_2 z^{-2} + \cdots + a_n z^{-n}} = \frac{B(z)}{A(z)} \quad (4-3)$$

MATLAB 信号处理工具箱提供了一个对 $X(z)$ 进行部分分式展开的函数 residuez，其语句格式为

$$[R,P,K]=residuez(B,A)$$

其中，B，A 分别表示 $X(z)$ 的分子与分母多项式的系数向量；R 为部分分式的系数向量；P 为极点向量；K 为多项式的系数。若 $X(z)$ 为有理真分式，则 K 为零。

【实例 4-3】 试用 MATLAB 命令对函数 $X(z) = \frac{18}{18 + 3z^{-1} - 4z^{-2} - z^{-3}}$ 进行部分分式

展开，并求出其 z 反变换。

解：MATLAB 源程序为

```
>>B=[18];
>>A=[18,3,-4,-1];
```

```
>>[R,P,K]=residuez(B,A)
```

```
R=
```

```
0.3600
```

```
0.2400
```

```
0.4000
```

```
P=
```

```
0.5000
```

```
-0.3333
```

```
-0.3333
```

```
K=
```

```
[]
```

从运行结果可知， $p_2 = p_3$ ，表示系统有一个二重极点。所以， $X(z)$ 的部分分式展开为

$$X(z) = \frac{0.36}{1-0.5z^{-1}} + \frac{0.24}{1+0.3333z^{-1}} + \frac{0.4}{(1+0.3333z^{-1})^2}$$

因此，其 z 反变换为

$$x(n) = [0.36 \times (0.5)^n + 0.24 \times (-0.3333)^n + 0.4(n+1)(-0.3333)^n]u(n)$$

4.1.2 系统函数的零极点分析

离散时间系统的系统函数定义为系统零状态响应的 z 变换与激励的 z 变换之比，即

$$H(z) = \frac{Y(z)}{X(z)} \quad (4-4)$$

如果系统函数 $H(z)$ 的有理函数表示式为

$$H(z) = \frac{b_1 z^m + b_2 z^{m-1} + \cdots + b_m z + b_{m+1}}{a_1 z^n + a_2 z^{n-1} + \cdots + a_n z + a_{n+1}} \quad (4-5)$$

那么，在 MATLAB 中系统函数的零极点就可通过函数 `roots` 得到，也可借助函数 `tf2zp` 得到，`tf2zp` 的语句格式为

$$[Z,P,K]=tf2zp(B,A)$$

其中， B 与 A 分别表示 $H(z)$ 的分子与分母多项式的系数向量。它的作用是将 $H(z)$ 的有理分式表示式转换为零极点增益形式，即

$$H(z) = k \frac{(z-z_1)(z-z_2)\cdots(z-z_m)}{(z-p_1)(z-p_2)\cdots(z-p_n)} \quad (4-6)$$

【实例 4-4】 已知一离散因果 LTI 系统的系统函数为

$$H(z) = \frac{z + 0.32}{z^2 + z + 0.16}$$

试用 MATLAB 命令求该系统的零极点。

解：用 tf2zp 函数求系统的零极点，MATLAB 源程序为

```
>>B=[1,0.32];  
>>A=[1,1,0.16];  
>>[R,P,K]=tf2zp(B,A)  
  
R=  
    -0.3200  
  
P=  
    -0.8000  
    -0.2000  
  
K=  
     1
```

因此，零点为 $z = 0.32$ ，极点为 $p_1 = 0.8$ 与 $p_2 = 0.2$ 。

若要获得系统函数 $H(z)$ 的零极点分布图，可直接应用 zplane 函数，其语句格式为

zplane(B,A)

其中，B 与 A 分别表示 $H(z)$ 的分子和分母多项式的系数向量。它的作用是在 Z 平面上画出单位圆、零点与极点。

【实例 4-5】 已知一离散因果 LTI 系统的系统函数为

$$H(z) = \frac{z^2 - 0.36}{z^2 - 1.52z + 0.68}$$

试用 MATLAB 命令绘出该系统的零极点分布图。

解：用 zplane 函数求系统的零极点，MATLAB 源程序为

```
>>B=[1,0,-0.36];  
>>A=[1,-1.52,0.68];  
>>zplane(B,A),grid on  
>>legend('零点','极点')  
>>title('零极点分布图')
```

程序运行结果如图 14-1 所示。可见，该因果系统的极点全部在单位圆内，故系统是稳定的。

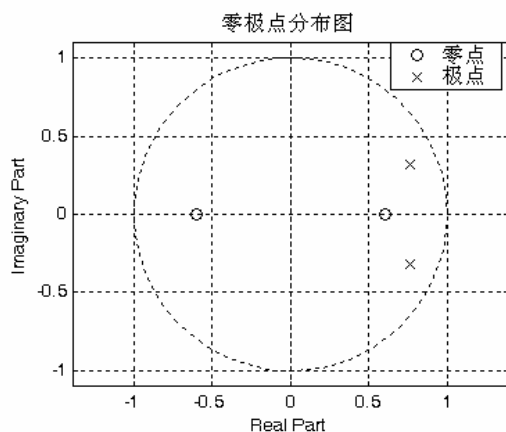


图 4-1 零极点分布图

4.1.3 系统函数的零极点分布与其时域特性的关系

与拉氏变换在连续系统中的作用类似,在离散系统中, z 变换建立了时域函数 $h(n)$ 与 z 域函数 $H(z)$ 之间的对应关系。因此, z 变换的函数 $H(z)$ 从形式可以反映 $h(n)$ 的部分内在性质。我们仍旧通过讨论 $H(z)$ 的一阶极点情况,来说明系统函数的零极点分布与系统时域特性的关系。

【实例 14-6】试用 MATLAB 命令画出现下列系统函数的零极点分布图、以及对应的时域单位取样响应 $h(n)$ 的波形,并分析系统函数的极点对时域波形的影响。

$$\begin{aligned}
 (1) \quad H_1(z) &= \frac{z}{z-0.8} & (2) \quad H_2(z) &= \frac{z}{z+0.8} & (3) \quad H_3(z) &= \frac{z}{z^2-1.2z+0.72} \\
 (4) \quad H_4(z) &= \frac{z}{z-1} & (5) \quad H_5(z) &= \frac{z}{z^2-1.6z+1} & (6) \quad H_6(s) &= \frac{z}{z-1.2} \\
 (7) \quad H_7(z) &= \frac{z}{z^2-2z+1.36}
 \end{aligned}$$

解: MATLAB 源程序为

```

>>b1=[1,0];
>>a1=[1,-0.8];
>>subplot(121)
>>zplane(b1,a1)
>>title('极点在单位圆内的正实数')
>>subplot(122)
>>impz(b1,a1,30);grid on;
>>figure
>>b2=[1,0];

```

```

>>a2=[1,0.8];
>>subplot(121)
>>zplane(b2,a2)
>>title('极点在单位圆内的负实数')
>>subplot(122)
>>impz(b2,a2,30);grid on;
>>figure
>>b3=[1,0];
>>a3=[1,-1.2,0.72];
>>subplot(121)
>>zplane(b3,a3)
>>title('极点在单位圆内的共轭复数')
>>subplot(122)
>>impz(b3,a3,30);grid on;
>>figure
>>b4=[1,0];
>>a4=[1,-1];
>>subplot(121)
>>zplane(b4,a4)
>>title('极点在单位圆上为实数 1')
>>subplot(122)
>>impz(b4,a4);grid on;
>>figure
>>b5=[1,0];
>>a5=[1,-1.6,1];
>>subplot(121)
>>zplane(b5,a5)
>>title('极点在单位圆上的共轭复数')
>>subplot(122)
>>impz(b5,a5,30);grid on;
>>figure
>>b6=[1,0];
>>a6=[1,-1.2];
>>subplot(121)
>>zplane(b6,a6)
>>title('极点在单位圆外的正实数')
>>subplot(122)
>>impz(b6,a6,30);grid on;

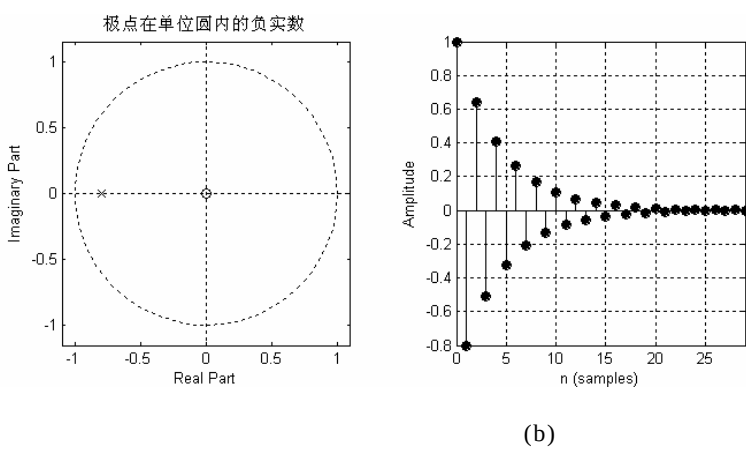
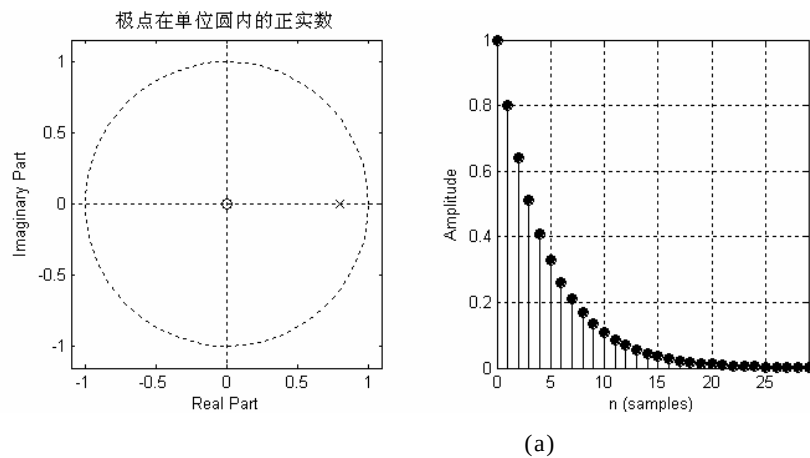
```

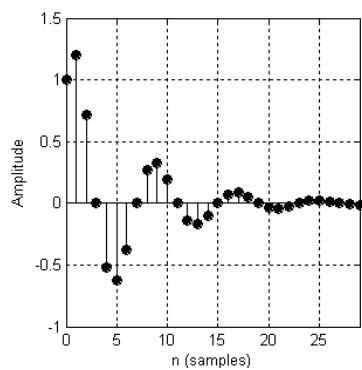
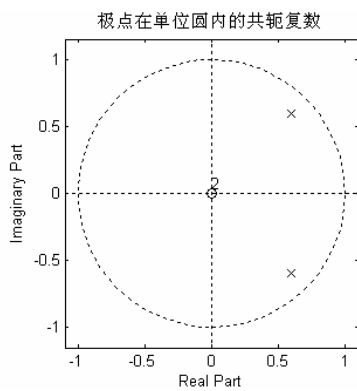
```

>>figure
>>b7=[1,0];
>>a7=[1,-2,1.36];
>>subplot(121)
>>zplane(b7,a7)
>>title('极点在单位圆外的共轭复数')
>>subplot(122)
>>impz(b7,a7,30);grid on;

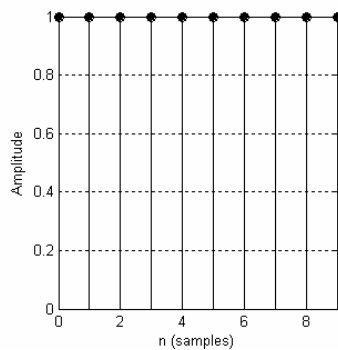
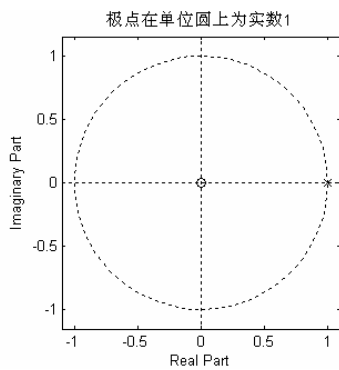
```

程序运行结果分别如图 14-2 的 (a)、(b)、(c)、(d)、(e)、(f)、(g) 所示。

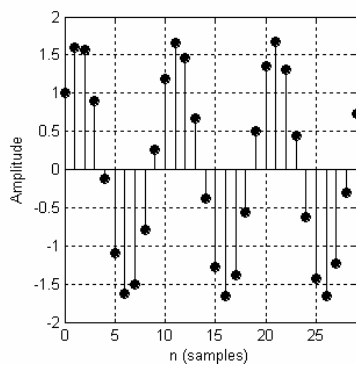
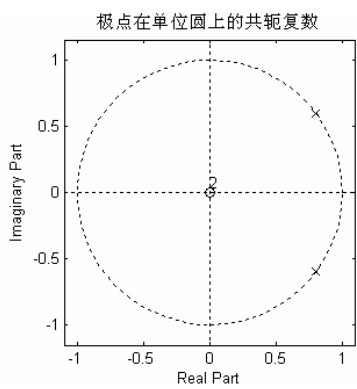




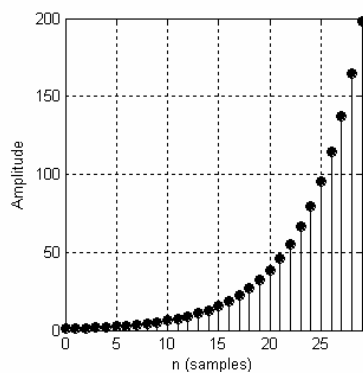
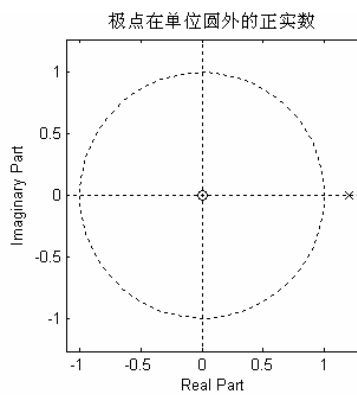
(c)



(d)



(e)



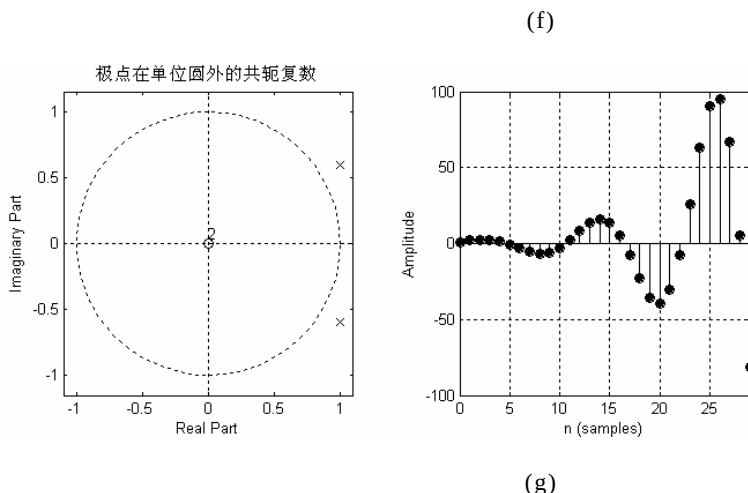


图 4-2 系统函数的零极点分布与其时域特性的关系

从图 14-2 可知，当极点位于单位圆内时， $h(n)$ 为衰减序列；当极点位于单位圆上时， $h(n)$ 为等幅序列；当极点位于单位圆外时， $h(n)$ 为增幅序列。若 $h(n)$ 有一阶实数极点，则 $h(n)$ 为指数序列；若 $h(n)$ 有一阶共轭极点，则 $h(n)$ 为指数振荡序列；若 $h(n)$ 的极点位于虚轴左边，则 $h(n)$ 序列按一正一负的规律交替变化。

4.1.4 离散时间 LTI 系统的频率特性分析

对于因果稳定的离散时间系统，如果激励序列为正弦序列 $x(n) = A\sin(n\omega)u(n)$ ，则系统的稳态响应为 $y_{ss}(n) = A|H(e^{j\omega})|\sin[n\omega + \varphi(\omega)]u(n)$ 。其中， $H(e^{j\omega})$ 通常是复数。离散时间系统的频率响应定义为

$$H(e^{j\omega}) = |H(e^{j\omega})|e^{j\varphi(\omega)} \quad (14-7)$$

其中， $|H(e^{j\omega})|$ 称为离散时间系统的幅频特性； $\varphi(\omega)$ 称为离散时间系统的相频特性；

$H(e^{j\omega})$ 是以 ω_s ($\omega_s = \frac{2\pi}{T}$ ，若零 $T=1$ ， $\omega_s = 2\pi$) 为周期的周期函数。因此，只要分析

$H(e^{j\omega})$ 在 $|\omega| \leq \pi$ 范围内的情况，便可分析出系统的整个频率特性。

MATLAB 提供了求离散时间系统频响特性的函数 `freqz`，调用 `freqz` 的格式主要有两种。一种形式为

$$[H, w] = \text{freqz}(B, A, N)$$

其中， B 与 A 分别表示 $H(z)$ 的分子和分母多项式的系数向量； N 为正整数，默认值为 512；返回值 w 包含 $[0, \pi]$ 范围内的 N 个频率等分点；返回值 H 则是离散时间系统频率

响应 $H(e^{j\omega})$ 在 $0 \sim \pi$ 范围内 N 个频率处的值。另一种形式为

$$[H,w]=\text{freqz}(B,A,N,'whole')$$

与第一种方式不同之处在于角频率的范围由 $[0, \pi]$ 扩展到 $[0, 2\pi]$ 。

【实例 4-6】用 MATLAB 命令绘制系统 $H(z) = \frac{z^2 - 0.96z + 0.9028}{z^2 - 1.56z + 0.8109}$ 的频率响应曲线。

解：利用函数 `freqz` 计算出 $H(e^{j\omega})$ ，然后利用函数 `abs` 和 `angle` 分别求出幅频特性与相频特性，最后利用 `plot` 命令绘出曲线。MATLAB 源程序为

```
>>b=[1 -0.96 0.9028];
>>a=[1 -1.56 0.8109];
>>[H,w]=freqz(b,a,400,'whole');
>>Hm=abs(H);
>>Hp=angle(H);
>>subplot(211)
>>plot(w,Hm),grid on
>>xlabel('\omega(rad/s)'),ylabel('Magnitude')
>>title('离散系统幅频特性曲线')
>>subplot(212)
>>plot(w,Hp),grid on
>>xlabel('\omega(rad/s)'),ylabel('Phase')
>>title('离散系统相频特性曲线')
```

程序运行结果如图 4-2 所示。

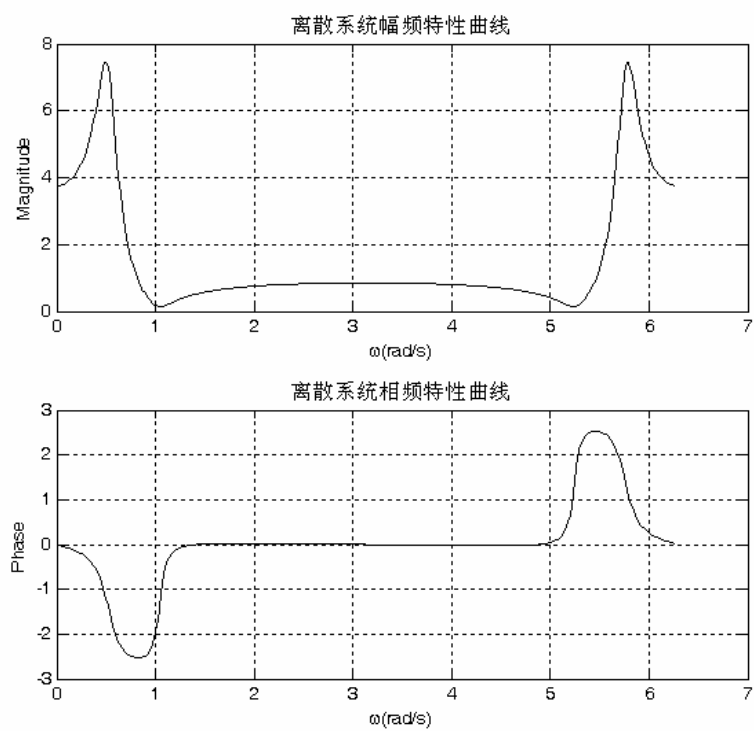


图 4-3 离散系统频响特性曲线

第 5 章 离散傅立叶变换及其快速算法

本章介绍了数字信号变换技术的详细内容，主要有：离散傅立叶变换、快速傅立叶变换（FFT）、离散余弦变换、Z 变换

5.1 离散傅立叶变换

傅立叶变换是信号分析和处理的重要工具。有限长序列作为离散信号的一种，在数字信号处理中占有极其重要的位置。对于有限长序列，离散傅立叶变换不仅在理论上有着重要的意义，而且有快速计算的方法——快速傅立叶变换。所以在各种数字信号处理的运算方法中，越来越起到核心的作用。

下面，就对离散傅立叶变换及其 MATLAB 函数应用，结合实际工程实例做说明。

5.1.1 傅立叶变换的几种形式

1、非周期连续时间信号的傅立叶变换

非周期连续时间信号 $x(t)$ 的傅立叶变换 $X(j\omega)$ 可以表示为

$$X(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt$$

逆变换为

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega$$

在这里， ω 是模拟角频率。可以看到，时域的连续函数造成频域的非周期谱，时域的非周期性造成频域连续谱。

结论：非周期连续时间函数对应于一非周期连续频域变换函数。

2、周期连续时间信号的傅立叶变换

周期为 T 的周期性连续时间信号 $x(t)$ 傅立叶变换是离散频域函数，可表示为

$$X(j\omega) = \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} x(t) e^{jm\omega t} d\omega$$

逆变换为

$$x(t) = \sum_{m=-\infty}^{\infty} X(jm\omega) e^{jm\omega t} d\omega$$

这就是经常称之为傅立叶级数的变换形式。在这里， ω 也是模拟角频率。可以看到，时域的连续函数造成频率域的非周期谱，频域函数的离散造成时域函数的周期性。

结论：周期连续时间函数对应于一非周期离散频域变换函数。

3、非周期离散时间信号 $x(n)$ 的傅立叶变换 $X(e^{j\omega})$ 可以表示为

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n) e^{-j\omega n}$$

逆变换为

$$x(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

在这里， ω 是数字频率，它和模拟角频率的关系为 $\omega = \Omega T$ 。可以看到，时域的取样对应于频域的周期延拓，而时域函数的非周期性造成频域的离散谱。

结论：非周期离散时间函数对应于一周期连续频域变换函数。

4、周期离散时间信号的傅立叶变换

周期离散时间信号 $x(n)$ 的傅立叶变换—离散傅立叶变换，可以表示为

$$X(k) = \sum_{n=0}^{N-1} e^{-j\frac{2\pi}{N}nk}$$

逆变换为

$$x(n) = \frac{1}{N} \sum_{K=0}^{N-1} X(k) e^{j\frac{2\pi}{N}nk}$$

可以看到，时域的取样对应于频域的周期延拓，而时域函数的周期性造成频域的离散谱。

结论：周期离散时间函数对应于一周期离散频域变换函数。

5.1.2 离散傅立叶变换

离散傅立叶级数变换是周期序列，仍不便于计算机计算。但离散傅立叶级数虽是周期序列，却只有 N 个独立的数值，所以它的许多特性可以通过有限长序列延拓来得到。对于一个长度为 N 的有限长序列 $x(n)$ ，也即 $x(n)$ 只在 $n=0 \sim (N-1)$ 个点上有非零值，其余皆为零，即

$$x(n) = \begin{cases} x(n), & 0 \leq n \leq N-1 \\ 0, & \text{其他} \end{cases}$$

把序列 $x(n)$ 以 N 为周期进行周期延拓得到周期序列 $\tilde{x}(n)$ ，则有

$$\tilde{x}(n) = \begin{cases} \tilde{x}(n), & 0 \leq n \leq N-1 \\ 0, & \text{其他} \end{cases}$$

所以，有限长序列 $x(n)$ 的离散傅立叶变换（DFT）为

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{-kn}, \quad 0 \leq n \leq N-1$$

逆变换为

$$x(n) = IDFT[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-kn}, \quad 0 \leq n \leq N-1$$

若将 DFT 变换的定义写成矩阵形式，则得到

$\mathbf{X} = \mathbf{A} \cdot \mathbf{x}$ ，其中 DFT 变换矩阵 $\mathbf{A}$ 为

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & W_N^1 & \dots & W_N^{N-1} \\ \dots & \dots & \dots & \dots \\ 1 & W_N^{N-1} & \dots & W_N^{(N-1)^2} \end{bmatrix}$$

Dftmtx 函数：用来计算 DFT 变换矩阵 $\mathbf{A}$ 的函数

调用方式

- (1) $\mathbf{A} = \text{dftmtx}(n)$ ：返回 $n \times n$ 的 DFT 变换矩阵 $\mathbf{A}$ 。若 $\mathbf{x}$ 为给定长度的行向量，则 $\mathbf{y} = \mathbf{x} * \mathbf{A}$ ，返回 $\mathbf{x}$ 的 DFT 变换 $\mathbf{y}$ 。
- (2) $\mathbf{A}_i = \text{conj}(\text{dftmtx}(n)) / n$ ：返回 $n \times n$ 的 IDFT 变换矩阵 $\mathbf{A}_i$ 。

应用说明

【实例 5-1】

```
>> A=dfmtx(4)
```

```
>> Ai=conj(dfmtx(4))/4
```

运行结果

A =

1.0000	1.0000	1.0000	1.0000
1.0000	0 - 1.0000i	-1.0000	0 + 1.0000i
1.0000	-1.0000	1.0000	-1.0000
1.0000	0 + 1.0000i	-1.0000	0 - 1.0000i

Ai =

0.2500	0.2500	0.2500	0.2500
0.2500	0 + 0.2500i	-0.2500	0 - 0.2500i
0.2500	-0.2500	0.2500	-0.2500
0.2500	0 - 0.2500i	-0.2500	0 + 0.2500i

【实例 5-2】如果 $x(n) = \sin(n\pi/8) + \sin(n\pi/4)$ 是一个 $N=16$ 的有限序列，用 MATLAB 求其 DFT 的结果，并画出其结果图，如图 5-1 所示。

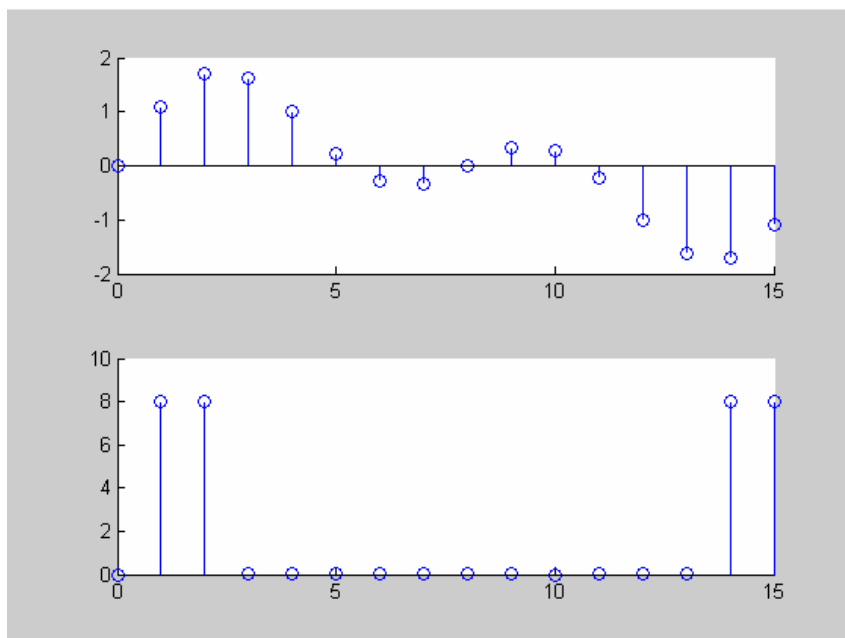


图 5-1 有限长序列的 DFT 结果图

程序

```
N=16;
n=0:1:N-1;      %时域采样
xn=sin(n*pi/8)+sin(n*pi/4);
k=0:1:N-1;      %频域采样
WN=exp(-j*2*pi/N);
nk=n'*k;
WNnk=WN.^nk;
Xk=xn*WNnk;
subplot(2,1,1)
stem(n,xn);
subplot(2,1,2)
stem(k,abs(Xk));
```

运算结果

Xk =

Columns 1 through 5

0.0000 -0.0000 - 8.0000i -0.0000 - 8.0000i 0.0000 - 0.0000i

0.0000 - 0.0000i

Columns 6 through 10

-0.0000 - 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i
0.0000 - 0.0000i

Columns 11 through 15

0.0000 - 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i 0.0000 - 0.0000i
0.0000 + 8.0000i

Column 16

0.0000 + 8.0000i

DFT 的性质

两个序列 $x_1(n)$ 和 $x_2(n)$ 都是 N 点有限长序列，设

$$X_1(k) = DFT[x_1],$$

$$X_2(k) = DFT[x_2]$$

1、线性

$DFT[ax_1(n) + bx_2(n)] = aX_1(k) + bX_2(k)$, 式中 a, b 为任意常数。

2、圆周移位

一个有限长序列 $x(n)$ 的圆周移位定义

$$x_m = x[(n+m)]_N R_N(n)$$

式中， $x[(n+m)]_N$ 表示 $x(n)$ 的周期延拓序列 $\tilde{x}(n)$ 的移位

$$x[(n+m)]_N = \tilde{x}(n+m)$$

有限长序列圆周移位后的 DFT 为

$$X_m(k) = DFT\{x_m[(n+m)]_N R_N(n)\} = W_N^{-kn} X(k)$$

【实例 5-3】 求有限长序列 $x(n) = 8(0.4)^n, 0 \leq n \leq 20$ 的圆周

移位 $x_m(n) = x[(n+10)]_{20} R_{20}(n)$ 。并画出其结果图，如图 5-2 所示。

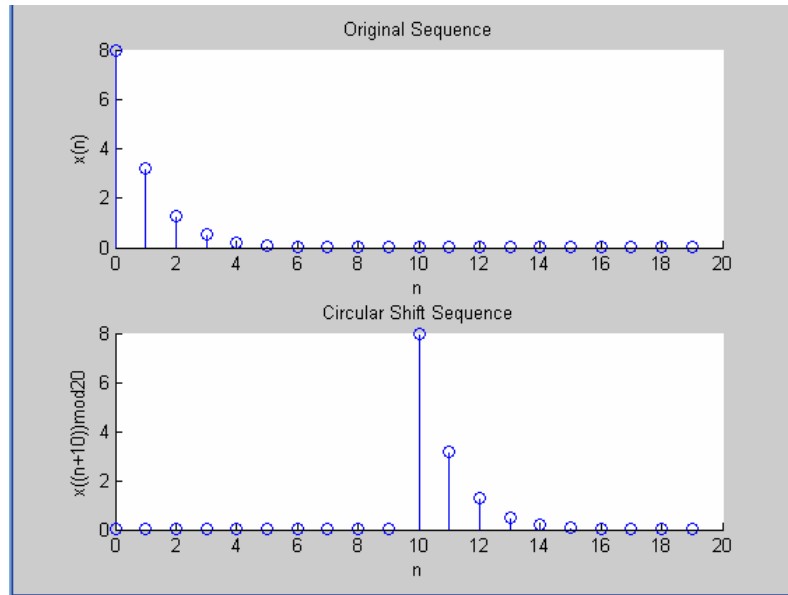


图 5-2 有限长序列的圆周移位结果图

程序

```

N=20;
m=10;
n=0:1:N-1;
x=8*(0.4).^n;
n1=mod((n+m),N);
xm=x(n1+1);
subplot(2,1,1)
stem(n,x);
title('Original Sequence');
xlabel('n');
ylabel('x(n)');
subplot(2,1,2)
stem(n,xm);
title('Circular Shift Sequence');
xlabel('n');
ylabel('x((n+10))mod20');

```

输出结果：

x =

Columns 1 through 8

8.0000	3.2000	1.2800	0.5120	0.2048	0.0819	0.0328	0.0131
--------	--------	--------	--------	--------	--------	--------	--------

Columns 9 through 16

0.0052	0.0021	0.0008	0.0003	0.0001	0.0001	0.0000	0.0000
--------	--------	--------	--------	--------	--------	--------	--------

Columns 17 through 20

0.0000	0.0000	0.0000	0.0000
--------	--------	--------	--------

3、圆周卷积

假设 $Y(k) = X_1(k)X_2(k)$

则有

$$Y(n) = IDFT[Y(k)] = \left[\sum_{m=0}^{N-1} x_1(m)x_2((n-m))_N \right] R_N(n) = \left[\sum_{m=0}^{N-1} x_2(m)x_1((n-m))_N \right] R_N(n)$$

用 $\otimes$ 表示圆周卷积，则上式可化简为

$$y(n) = IDFT[X_1(k)X_2(k)] = x_1(n) \otimes x_2(n) = x_2(n) \otimes x_1(n)$$

MATLAB 内部用于计算圆周卷积的函数 Circonv 程序如下：

```
function y=circonvt(x1,x2,N)
if(length(x1)>N)
    error('N should bigger than or equal to the length of x1!')
end
if(length(x2)>N)
    error('N should bigger than or equal to the length of x2!')
x1=[x1,zeros(1,N-length(x1))]
x2=[x2,zeros(1,N-length(x2))]
m=[0:1:N-1]
for n=1:1:N
end
y=x1*H'
```

4.共轭对称性

令 $x(n)$ 的共轭复数序列为 $x^*(n)$ ，则

$$DFT[x^*(n)] = X^*[(N-K)]_N$$

用 $x_r(n)$ 和 $x_i(n)$ 分别表示序列 $x(n)$ 的实部和虚部，即

$$x(n) = x_r(n) + jx_i(n)$$

$$\begin{cases} x_r(n) = \frac{1}{2}[x(n) + x^*(n)] \\ x_i(n) = \frac{1}{2j}[x(n) - x^*(n)] \end{cases}$$

用 $X_R(k)$ 和 $X_I(k)$ 分别表示实部和虚部序列的 DFT ，即

$$X_R(k) = DFT[x_r(n)]$$

$$X_I(k) = DFT[x_i(n)]$$

而且可以证明得到

$$X_R(k) = X_R^*[(N-K)]_N$$

$$X_I(k) = -X_I^*[(N-K)]_N$$

通常称 $X_R(k)$ 为 $X(k)$ 的共轭偶部， $X_I(k)$ 为 $X(k)$ 的共轭奇部。所以说，对于时域、频域的 DFT 对应关系来说，序列 $x(n)$ 实部对应于 $X(k)$ 的共轭偶部，序列 $x(n)$ 的虚部对应于 $X(k)$ 的共轭奇部。

5. 序列乘积

$$DFT[x_1(n)x_2(n)] = \frac{1}{N} X_1(k) \otimes X_2(k)$$

6. DFT 形式下的帕塞瓦尔定理

$$\sum_{n=0}^{N-1} x(n)y^*(n) = \sum_{k=0}^{N-1} X(k)Y^*(k)$$

5.2 快速傅立叶变换 (FFT)

在信号处理中， DFT 的计算具有举足轻重的地位，信号的相关、滤波、谱估计等

都要通过 DFT 来实现。然而，当 N 很大的时候，求一个 N 点的 DFT 要完成 $N \times N$ 次复数乘法和 $N(N-1)$ 次复数加法，其计算量相当大。1965 年 J.W.Cooley 和 J.W.Tukey 巧妙地利用 W_N 因子的周期性和对称性，构造了一个 DFT 快速算法，即快速傅立叶变换(FFT)。

5.4.1 函数应用

MATLAB 为计算数据的离散快速傅立叶变换，提供了一系列丰富的数学函数，主要有 Fft、Ifft、Fft2、Ifft2、Fftn、ifftn 和 Fftshift、Ifftshift 等。当所处理的数据的长度为 2 的幂次时，采用基-2 算法进行计算，计算速度会显著增加。所以，要尽可能使所要处理的数据长度为 2 的幂次或者用添零的方式来添补数据使之成为 2 的幂次。

1. Fft 和 Ifft 函数

调用方式

$$(1) \quad Y = \text{fft}(X)$$

参数说明

- 如果 X 是向量，则采用傅立叶变换来求解 X 的离散傅立叶变换；
- 如果 X 是矩阵，则计算该矩阵每一列的离散傅立叶变换；
- 如果 X 是 (N^*D) 维数组，则是对第一个非单元元素的维进行离散傅立叶变换；

$$(2) \quad Y = \text{fft}(X, N)$$

参数说明

N 是进行离散傅立叶变换的 X 的数据长度，可以通过对 X 进行补零或截取来实现。

$$(3) \quad (3) \quad Y = \text{fft}(X, [], \text{dim}) \text{ 或 } Y = \text{fft}(X, N, \text{dim})$$

参数说明

- 在参数 dim 指定的维上进行离散傅立叶变换；
- 当 X 为矩阵时， dim 用来指定变换的实施方向： $\text{dim}=1$, 表明变换按列进行； $\text{dim}=2$ 表明变换按行进行。

函数 Ifft 的参数应用与函数 Fft 完全相同。

应用说明

【实例 5-4】fft 的应用

```
X=[2 1 2 8];
Y=fft(X)
```

运行结果

```
Y=
13.0000      0+7.0000i    -5.0000      0-7.0000i
```

【实例 5-5】fft(X,N,dim)的应用

```
A=[2 5 7 8;
1 4 0 5;
3 8 5 1;
9 1 2 7];

Z=fft(A,[],1)
```

```
Z =
15.0000      18.0000      14.0000      21.0000
-1.0000 + 8.0000i  -3.0000 - 3.0000i   2.0000 + 2.0000i   7.0000 + 2.0000i
-5.0000      8.0000      10.0000     -3.0000
-1.0000 - 8.0000i  -3.0000 + 3.0000i   2.0000 - 2.0000i   7.0000 - 2.0000i
```

【实例 5-6】fft 在信号分析中的应用

使用频率分析方法从受噪声污染的信号 x(t)中鉴别出有用的信号。

程序：

```
t=0:0.001:1;    %采样周期为 0.001s,即采样频率为 1000Hz;
                %产生受噪声污染的正弦波信号;
x=sin(2*pi*100*t)+sin(2*pi*200*t)+rand(size(t));
subplot(2,1,1)
plot(x(1:50));    %画出时域内的信号;
Y=fft(x,512);    %对 X 进行 512 点的傅立叶变换;
f=1000*(0:256)/512;    %设置频率轴（横轴）坐标，1000 为采样频率;
```

```
subplot(2,1,2)
plot(f,Y(1:257)); %画出频域内的信号
```

运行结果

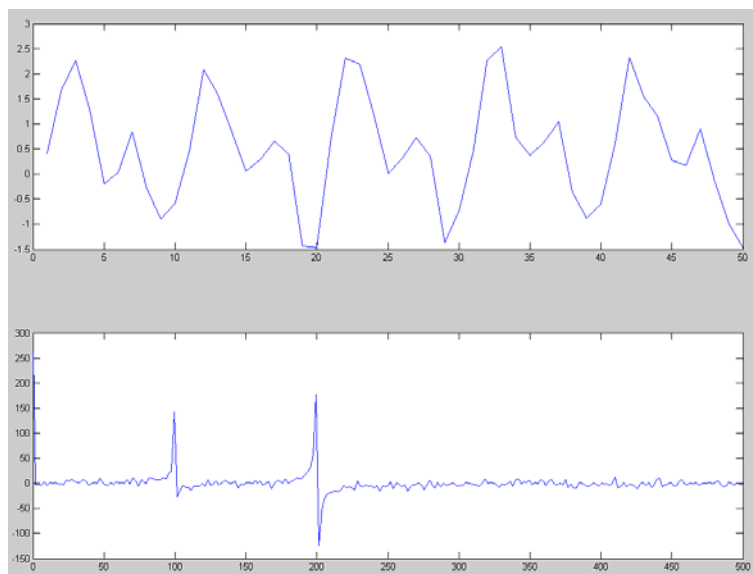


图 5-3 时域信号和频域信号的比较

由上面的图 5-3 可以看出，从受噪声污染信号的时域形式中，很难看出正弦波的成分。但是通过对 $x(t)$ 作傅立叶变换，把时域信号变换到频域进行分析，可以明显看出信号中 100Hz 和 200Hz 的两个频率分量。

2. Fft2 和 Ifft 函数

调用方式

(1) $Y = \text{fft2}(X)$

参数说明

- 如果 X 是向量，则此傅立叶变换即变成一维傅立叶变换 fft ;
- 如果 X 是矩阵，则是计算该矩阵的二维快速傅立叶变换;
- 数据二维傅立叶变换 $\text{fft2}(X)$ 相当于 $\text{fft}(\text{fft}(X)')$ ，即先对 X 的列做一维傅立叶变换，然后再对变换结果的行做一维傅立叶变换。

(2) $Y = \text{fft2}(X, M, N)$

通过对 X 进行补零或截断，使得成为 (M*N) 的矩阵。

函数 Ifft2 的参数应用与函数 Fft2 完全相同。

Fftn.、Ifftn 是对数据进行多维快速傅立叶变换，其应用与 Fft2、Ifft2 类似；在此，不再一

一赘述。

【实例 5-7】fft2、ifft2 的应用

```
A=[2 5 7 8 9;  
    1 3 7 5 0;  
    2 6 1 4 9;  
    8 1 5 2 6];  
Y=fft2(A);  
B=ifft2(Y);
```

运行结果：

Y =				
91.0000	-6.5000 + 7.9717i	-6.5000 + 6.2411i	-6.5000 - 6.2411i	-6.5000 - 7.9717i
9.0000 + 6.0000i	-15.4199 +12.0566i	-1.7542 + 4.7087i	9.5525 - 0.2711i	-1.3785 +12.5057i
15.0000	0.0623 +10.0453i	-20.0623 - 5.6206i	-20.0623 + 5.6206i	0.0623 -10.0453i
9.0000 - 6.0000i	-1.3785 -12.5057i	9.5525 + 0.2711i	-1.7542 - 4.7087i	-15.4199 -12.0566i

B =				
2.0000	5.0000	7.0000	8.0000	9.0000
1.0000	3.0000	7.0000	5.0000	0
2.0000	6.0000	1.0000	4.0000	9.0000
8.0000	1.0000	5.0000	2.0000	6.0000

3. Fftshift 和 Ifftshift 函数

调用方式

```
Z=fftshift(Y)
```

此函数可用于将傅立叶变换结果 Y (频域数据) 中的直流成分 (即频率为 0 处得值) 移到频谱的中间位置。

参数说明

- 如果 X 是向量，则变换 Y 的左右两边；

- 如果 X 是矩阵，则交换 Y 的一、三象限和二、四象限；
- 如果 Y 是多维数组，则在数组的每一维交换其“半空间”。

函数 Ifftshift 的参数应用与函数 Fftshift 完全相同。

【实例 5-8】 fftshift 的应用

```
X=rand(5,4);
```

```
y=fft(X);
```

```
z=fftshift(y);%只将傅立叶变换结果 y 中的直流成分移到频谱的中间位置.
```

运行结果：

y=

3.2250	2.5277	1.4820	1.6314
0.3294+0.2368i	0.0768+0.3092i	0.6453+0.4519i	-0.7240-0.4116i
-0.2867-0.6435i	0.5657+0.4661i	-0.5515+0.2297i	-0.0573-0.0881i
-0.2867+0.6435i	0.5657-0.4661i	-0.5515-0.2297i	-0.0573+0.0881i
0.3294-0.2368i	0.0768-0.3092i	0.6453-0.4519i	-0.7240+0.4116i

Z=

-0.5515-0.2297i	-0.0573+0.0881i	-0.2867+0.6435i	0.5657-0.4661i
0.6453-0.4519i	-0.7240+0.4116i	0.3294-0.2368i	0.0768-0.3092i
1.4820	1.6314	3.2250	2.5277
0.6453+0.4519i	-0.7240-0.4116i	0.3294+0.2368i	0.0768+0.3092i
-0.5515+0.2297i	-0.0573-0.0881i	-0.2867-0.6435i	0.5657+0.4661i

5.2.2 离散傅立叶变换（IFFT）

逆变换 IFFT 为

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad r = 0, 1, \dots, \frac{N}{2} - 1$$

通过下列修改，就可以用 FFT 算法来实现逆 FFT 运算：

- 增加一个归一化因子 $\frac{1}{N}$ ；
- 将用 W_N^{nk} 其复数共轭 W_N^{-nk} 代替

但是由因为，

$$x(n) = \frac{1}{N} \left[\sum_{k=0}^{N-1} X^*(k) W_N^{nk} \right]^* = \frac{1}{N} \{FFT [X^*(k)]\}^*$$

所以，求 $X(k)$ 的逆 FFT 可以分为以下 3 个步骤：

- (1) 取 $X(k)$ 的共轭，得到 $X^*(k)$ ；
- (2) 求 $X^*(k)$ 的 FFT，得 $Nx^*(k)$ ；
- (3) 取 $x^*(k)$ 的共轭，并除以 N ，即得到 $x(n)$ 。

采用这种方法可以直接用 FFT 程序来计算逆 FFT。有关 IFFT 的具体应用，与 FFT 一一对应，在此不再赘述。

第 6 章 IIR 数字滤波器的 MATLAB 实现

6.1 实验原理及实例分析

6.1.1 IIR 数字滤波器的传递函数及特点

设 IIR 滤波器的输入序列为 $x(n)$ ，则 IIR 滤波器的输入序列 $x(n)$ 与输出序列 $y(n)$ 之间的关系可以用下面的方程式表示：

$$y(n) = \sum_{i=0}^M b_i x(n-i) + \sum_{j=1}^N a_j x(n-j)$$

其中， a_j 和 b_i 是滤波器的系数，其中 a_j 中至少有一个非零。与之相对应的差分方程

为：

$$H(Z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + \dots b_M z^{-M}}{1 - a_1 z^{-1} - \dots a_N z^{-N}}$$

由传递函数可以发现无限常单位冲激响应滤波器有如下特点：

- (1) 单位冲激响应 $h(n)$ 是无限长的。
- (2) 系统传递函数 $H(z)$ 在有限 z 平面上有极点存在。
- (3) 结构上存在着输出到输入的反馈，也就是结构上是递归型的。

6.1.2 IIR 数字滤波器的设计与实现

IIR 数字滤波器的设计有多种方法，如频率变换法、数字域直接设计以及计算辅助设计等。下面只介绍频率变换设计法。首先考虑由模拟低通滤波器到数字低通滤波器的转换，其基本的设计过程如下：

- (1) 将数字滤波器的技术指标转换为模拟滤波器的技术指标；
- (2) 设计模拟滤波器 $G(S)$ ；
- (3) 将 $G(S)$ 转换成数字滤波器 $H(Z)$ ；

在低通滤波器的设计基础上，可以得到数字高通、带通、带阻滤波器的设计流程如下：

- (1) 给定数字滤波器的设计要求（高通、带阻、带通）；
- (2) 转换为模拟（高通、带阻、带通）滤波器的技术指标；
- (3) 转换为模拟低通滤波器的指标；

- (4) 设计得到满足第三步要求的低通滤波器传递函数；
- (5) 通过频率转换得到模拟（高通、带阻、带通）滤波器；
- (6) 变换为数字（高通、带阻、带通）滤波器。

在 matlab 中设计 IIR 滤波器的方法及其它它们所用到的函数如表 6－1 所示。

表 6－1 matlab 中设计 IIR 滤波器的方法列表

方法	描述	函数
模拟原型法	采用经典低通滤波器作为连续域上的设计模型，通过频率变换得到 IIR 数字滤波器，最后进行离散化处理	完整设计函数： Beself,butter,cheby1,cheby2,ellip 滤波器的阶估计函数： Buttord,cheb1ord,cheb2ord,ellipord 低通模拟滤波器原型函数： beselap,buttap,cheb1ap,cheb2ap,ellipap 频域变换函数： Lp2bp,lp2bs,lp2hp,lp2lp 其他函数： Bilinear,impinvar
直接设计方法	直接在离散时域上估计线性的幅度响应	yulewalk
通用 butterworth 设计方法	使用 butterworth 设计低通数字滤波器	Maxflat
参数建模方法	寻找接近于所需要设计的滤波器的通用原型	时域上的建模函数： Lpc,prony,stmcb 频域上的建模函数： Invfreqs,invfreqz

表 6－2 频率转换函数列表

频率转换	转换函数
低通到低通 $s' = s / \omega_0$	[numt,dent]=lp2lp(num.den,w0) [At,Bt,Ct,Dt]=lp2lp(A,B,C,D,w0)
低通到高通 $s' = \omega_0 / s$	[numt,dent]=lp2hp(num.den,w0) [At,Bt,Ct,Dt]=lp2hp(A,B,C,D,w0)
低通到带通 $s' = \frac{\omega_0}{B_\omega} \frac{(s/\omega_0)^2 + 1}{s/\omega_0}$	[numt,dent]=lp2bp(num.den,w0) [At,Bt,Ct,Dt]=lp2bp(A,B,C,D,w0)

低通到带阻 $s' = \frac{B_\omega}{\omega_0} \frac{s/\omega_0}{(s/\omega_0)^2 + 1}$	$[\text{numt}, \text{dent}] = \text{lp2bs}(\text{num}, \text{den}, \text{w0})$ $[\text{At}, \text{Bt}, \text{Ct}, \text{Dt}] = \text{lp2bs}(\text{A}, \text{B}, \text{C}, \text{D}, \text{w0})$
--	--

6.1.3 标准数字滤波器设计函数

Matlab 提供了一组标准的数字滤波器设计函数，大大简化了滤波器的设计过程。

1、butter

功能：Butterworth 模拟/数字滤波器设计

格式：[b, a]=butter(n, wn, 'ftype', 's')

[b, a]=butter(n, wn, 'ftype')

说明：

- 选项中加入 'S' 用于设计各种模拟 Butterworth 滤波器；不加设计各种数字 Butterworth 滤波器
- Ftype 为缺省，设计低通滤波器
- Ftype=high, 设计高通滤波器
- Ftype=stop, 设计带阻滤波器

【实例 6-1】设计一个 5 阶 Butterworth 数字高通滤波器，阻带截止频率为 250Hz。设采样频率为 1000Hz。

解：源代码如下：

```
[b, a]=butter(5, 250/500, 'high')
[z, p, k]=butter(5, 250/500, 'high')
freqz(b, a, 512, 1000)
```

程序运行后，产生结果如下所示。

```
b =
    0.0528    -0.2639     0.5279    -0.5279     0.2639    -0.0528

a =
    1.0000    -0.0000     0.6334    -0.0000     0.0557    -0.0000

z =
    1     1     1     1     1

p =
    0.0000 + 0.7265i    0.0000 - 0.7265i    0.0000 + 0.3249i
    0.0000 - 0.3249i    0.0000

k =
    0.0528
```

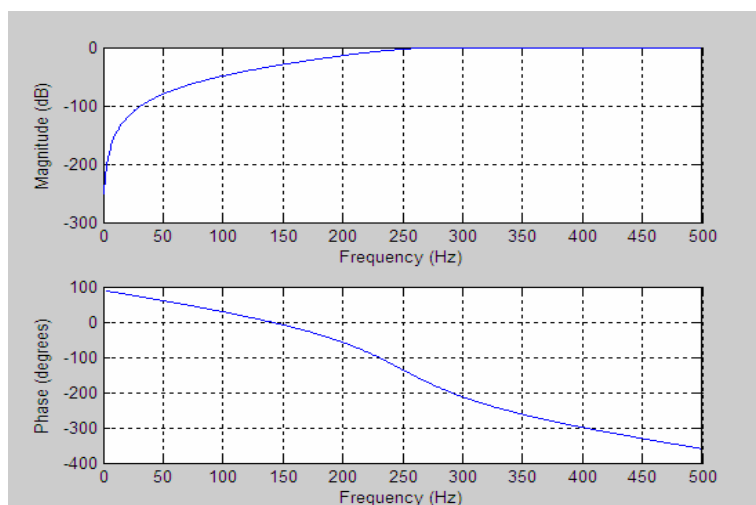


图 6-1 5 阶 Butterworth 数字高通滤波器

2、Cheby1、Cheby2

功能：chebyshevI、chebyshevII 型模拟/数字滤波器设计

格式：[b, a]= cheby1(n, Rp, wn, 'ftype',)

[b, a]= cheby2(n, Rs, wn, 'ftype')

【实例 6-2】设计一个 7 阶 chebyshevII 型数字低通滤波器，截止频率为 3000Hz，
Rs=30dB。设采样频率为 1000Hz。

解：源程序如下：

```
[b, a]=cheby2(7, 30, 300/500');
[z, p, k]=butter(5, 250/500, 'high');
freqz(b, a, 512, 1000)
```

程序运行后，产生如图 6-2 所示的波形。

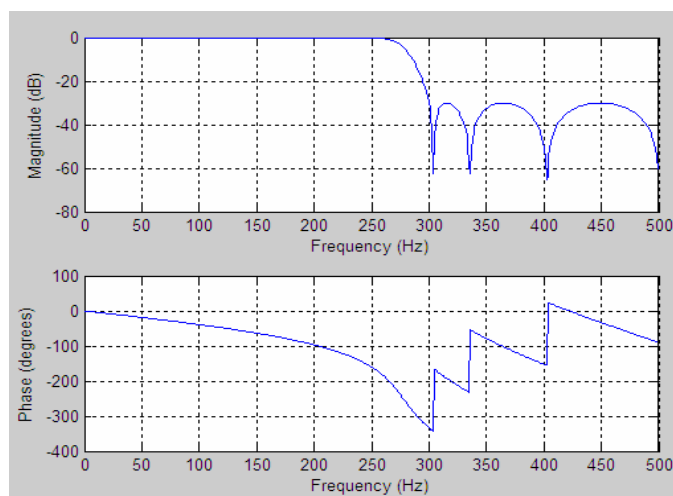


图 6-2 7 阶 chebyshevII 型数字低通滤波器

6.1.4 冲激响应不变法

一般来说，在要求时域冲激响应能模仿模拟滤波器的场合，一般使用冲激响应不变法。冲激响应不变法一个重要特点是频率坐标的变换是线性的，因此如果模拟滤波器的频响带限于折叠频率的话，则通过变换后滤波器的频响应可不失真的反映原响应与频率的关系。

【实例 6-3】设计一个中心频率为 500Hz，带宽为 600Hz 的数字带通滤波器，采样频率为 1000Hz。

解：源代码如下：

```
[z, p, k]=buttap(3);  
[b, a]=zp2tf(z, p, k);  
[bt, at]=lp2bp(b, a, 500*2*pi, 600*2*pi);  
[bz, az]=impinvar(bt, at, 1000); %将模拟滤波器变换成数字滤波器  
freqz(bz, az, 512, 'whole', 1000)
```

程序运行后，产生如图 6-3 所示的波形。

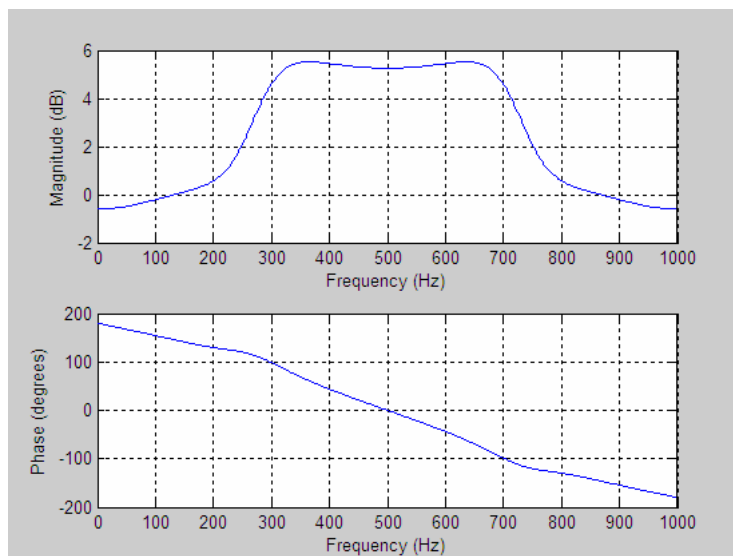


图 6-3 用冲激响应不变法设计数字低通滤波器

6.1.5 双线性变换法

与冲激响应不变法比较，双线性变换的主要优点是靠频率的非线性关系得到 S 平面与 Z 平面的单值一一对应关系，整个值对应于单位圆一周。所以从模拟传递函数可直接通过代数置换得到数字滤波器的传递函数。

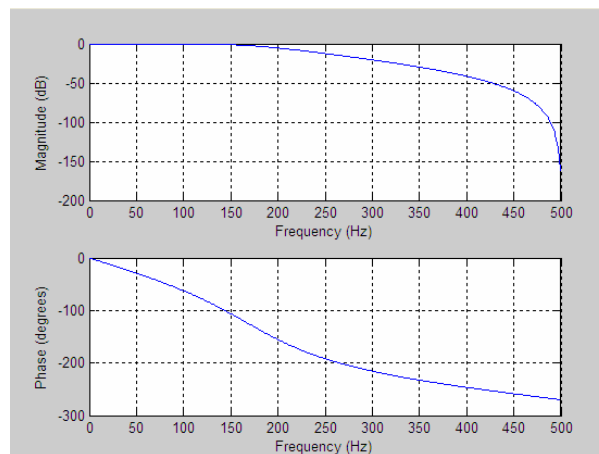
【实例 6-4】设计一个截止频率为 200Hz 的数字低通滤波器，采用频率为 1000Hz。

解：源代码如下：

```
[z, p, k]=buttap(3);  
[b, a]=zp2tf(z, p, k);  
[bt, at]=lp2lp(b, a, 200*2*pi);  
[bz, az]=bilinear(bt, at, 1000);  
freqz(bz, az, 512, 1000)
```

结果如图 6-4 所示：

```
bz =  
    0.0753    0.2259    0.2259    0.0753  
az =  
    1.0000   -0.8266    0.5154   -0.0865
```



【实例 6-5】基于 Butterworth 模拟滤波器原型，使用双线性转换设计数字滤波器，其

中参数指标为：图 6-4 用双线性变换法设计数字低通滤波器

通带截止频率： $\Omega_p = 0.2\pi$ 通带波动值： $R_p = 1dB$

阻带截止频率： $\Omega_s = 0.3\pi$ 阻带波动值： $A_s = 15dB$

解：首先确定滤波器的阶数 N ，同时根据 Ω_p 和 Ω_s 确定 $\Omega_c = 0.5$ 。

接着使用 `bilinear` 进行双线性转换，最后绘制在频域上的各种图像，其源代码如下：

% 数字滤波器指标

```
wp=0.2*pi;
```

```
ws=0.3*pi;
```

```
Rp=1;
```

```
As=15;
```

% 将数字滤波器指标反转变为模拟滤波器的参数

```
T=1;
```

```

fs=1/T;
omegap=(2/T)*tan(wp/T);
omegas=(2/T)*tan(ws/T);
ep=sqrt(10^(Rp/10)-1);
Ripple=sqrt(1/(1+ep*ep));
Attn=1/(10^(As/20));
%butterworth 原型模拟滤波器的设计
[cs,ds]=afd_butt(omegap,omegas,Rp,As);
% 双线性变换
[b,a]= bilinear(cs,ds,T,fs);
% 频域图像的绘制
freqz(b,a);

```

程序运行后，产生 4 阶的 butterworth 数字滤波器，频率响应如图 6—5 所示的波形。

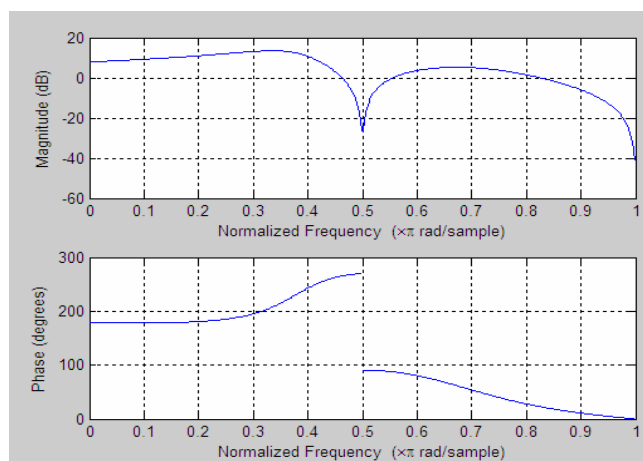


图 6—5 4 阶 Butterworth 数字滤波器

第 7 章 FIR 数字滤波器

7.1 实验原理及实例分析

7.1.1 FIR 低通数字滤波器的设计原理

如果系统的冲激响应 $h_d(n)$ 为已知，则系统的输入/输出关系为：

$$y(n) = x(n) \cdot h_d(n)$$

对于低通滤波器，只要设计出低通滤波器的冲激响应函数，就可以由上式得到系统的输出了。假设所希望的数字滤波器的频率响应为 $H_d(e^{j\omega})$ ，它是频域的周期函数，周期为 2π ，那么它与 $H_d(e^{j\omega})$ 相对应的傅立叶系数为

$$h_d(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} H_d(e^{j\omega}) e^{jn\omega} d\omega$$

以 $h_d(n)$ 为冲激响应的数字滤波器将具有频域响 $H_d(e^{j\omega})$ 。

但是将 $h_d(n)$ 作为滤波器脉冲响应有两个问题：

- (1) 它是无限长的，与 FIP 滤波器脉冲响应有限长这一前提不一致
- (2) 它是非因果的， $h_d(n) \neq 0, n < 0$

对此，要采取以下的措施，

- (1) 将 $h_d(n)$ 截短
- (2) 将其往右平移，

由此得到 $h_2(n)$ 的实际频域响应 $H_d(e^{j\omega}) = \sum_{n=0}^{N-1} h_2(n) e^{jn\omega}$ ，与理想频域响应 $H_d(e^{j\omega})$ 相近，

但不完全一致。理论证明上述现象是对 $h_d(n)$ 进行简单截短处理的必然结果，一般称为

吉布斯现象，为尽可能的减少吉布斯现象，应对 $h_d(n)$ 进行加窗截取，即以

$h(n) = h_d(n) \cdot W_N(n)$ 作为 FIR 滤波器的系数。

常用的窗函数有矩形窗、海明窗和布莱克曼窗等。

7.1.2 用窗函数法设计 FIR 滤波器

Matlab 设计 FIR 滤波器有多种方法和对应的函数，见表 7-1。

表 7-1 matlab 设计 FIR 滤波器的方法和函数

方法	描述	函数
窗方法	使用窗函数和逆傅立叶变换实现	fir1,fir2,kaiserord 等
多带方法	包含子带频率域	firls, remez 等
最小二乘法	使用最小二乘法将整个频率域上的错误几率压缩到最小	fircls , fircls1 等
任意响应法	使用任意响应，包括非线性相位以及复滤波器	cremez 等
余弦法	使用三角函数的低通响应	firrcos 等

窗函数方法不仅在数字滤波器的设计中占有重要的地位，同时可以用于功率谱的估计，从根本上讲，使用窗函数的目的就是消除由无限序列的截短而引起的 Gibbs 现象所带来的影响。

窗函数设计线性相位 FIR 滤波器步骤如下：

- (1) 确定数字滤波器的性能要求，临界频率 $\{w_k\}$ ，滤波器单位脉冲响应长度 N
- (2) 根据性能要求，合理选择单位脉冲响应 $h(n)$ 的奇偶对称性，从而确定理想频率响应 $H_d(e^{jw})$ 的幅频特性和相频特性
- (3) 求理想单位脉冲响应 $h_d(n)$ ，在实际计算中，可对 $H_d(e^{jw})$ 采样，并对其求 IDFT 的 $h_M(n)$ ，用 $h_M(n)$ 代替 $h_d(n)$
- (4) 选择适当的窗函数 $w(n)$ ，根据 $h(n) = h_d(n) \cdot W_N(n)$ 求所需设计的 FIR 滤波器单位脉冲响应
- (5) 求 $H_d(e^{jw})$ ，分析其幅频特性，若不满足要求，可适当改变窗函数形式或长度 N ，重复上述设计过程，以得到满意的结果

【实例 7-1】 设计一个 34 阶的高通滤波器，截止频率为 0.48，使用具有 30dB 波纹的 chebyshev 窗。

解：源程序如下：

```
b=fir1(34,0.48,'high',chebwin(35,30));
```

freqz(b, 1, 512)

其响应波形如图 7-1 所示。

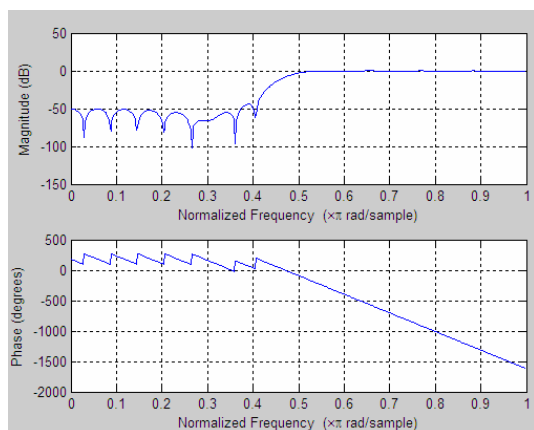


图 7-1 带通 FIR 滤波器

【实例 7-2】 设计具有下面指标的低通 FIR 滤波器

$$w_p = 0.2\pi, R_p = 0.25dB,$$

$$w_s = 0.3\pi, A_s = 50dB$$

由于其最小阻带衰减为 50dB，因此可以选择 hamming 窗来实现这个滤波器，因为它具有较小的过渡带。

解：MATLAB 源程序为

% 数字滤波器指标

wp=0.2*pi;

ws=0.3*pi;

tr_width=ws-wp;

M=ceil(6.6*pi/tr_width)+1;

n=[0:1:M-1];

wc=(ws+wp)/2;

hd=ideal_lp(wc,M);

% 生成 hamming 窗

w_ham=(hamming(M))';

% 频域图像的绘制

h=hd.*w_ham;

freqz(h,[1])

figure(2);

```

subplot(2,2,1),stem(n,hd);title('理想脉冲响应')
axis([0 M-1 -0.3 0.3]);xlabel('n');ylabel('hd(n)')
xa=0.*n;
hold on
plot(n,xa,'k');
hold off

```

```

subplot(2,2,2),stem(n,w_ham);title('hamming 窗')
axis([0 M-1 -0.3 1.2]);xlabel('n');ylabel('w(n)')

```

```

subplot(2,2,3),stem(n,h);title('实际脉冲响应')
axis([0 M-1 -0.3 0.3]);xlabel('n');ylabel('h(n)')
hold on
plot(n,xa,'k');
hold off

```

其响应波形如图 7-2 所示。

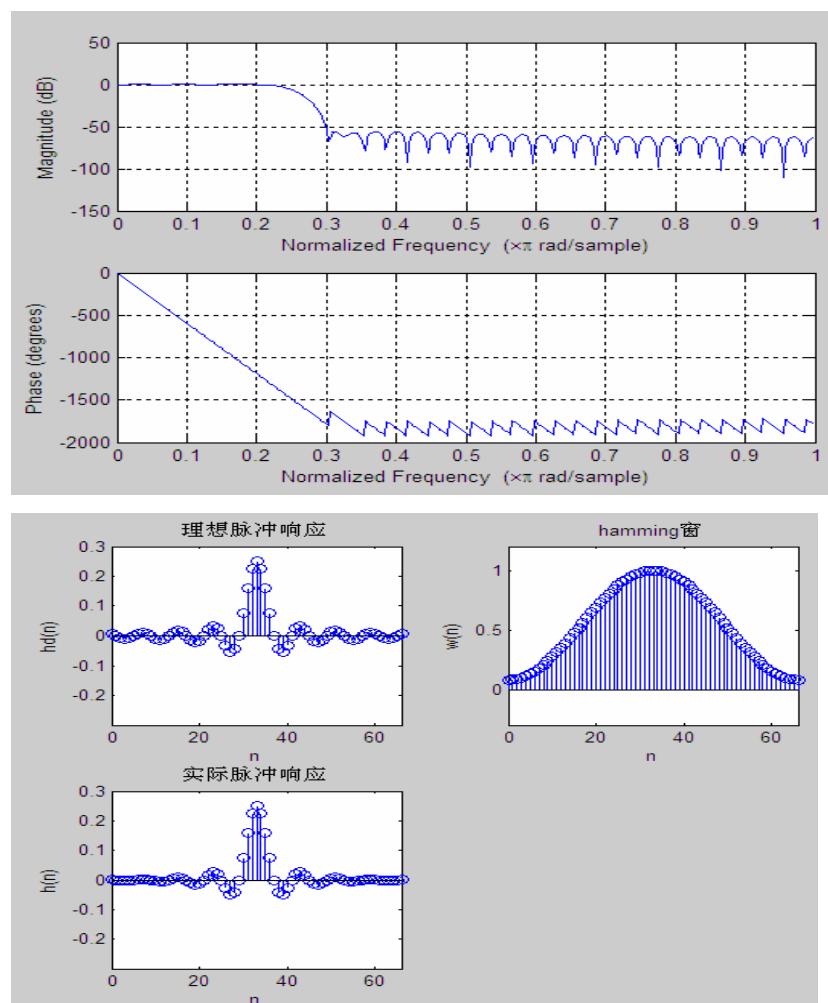


图 7-2 hamming 窗函数设计 FIR 滤波器

【实例 7-3】设带通滤波器的指标为

$$w_{p1} = 0.35\pi, w_{p2} = 0.8\pi,$$
$$w_{s1} = 0.2\pi, w_{s2} = 0.65\pi, A_s = 60\text{dB}$$

选择 Blackman 窗来实现这个滤波器。

解：MATLAB 源程序为

% 数字滤波器指标

```
ws1=0.2*pi; wp1=0.35*pi;
ws2=0.65*pi; wp2=0.8*pi;
As=60;
tr_width=min((wp1-ws1),(wp2-ws2));
M=ceil(11*pi/tr_width)+1;
n=[0:1:M-1];
wc1=(ws1+wp1)/2;
wc2=(ws2+wp2)/2;
hd=ideal_lp(wc2,M)-ideal_lp(wc1,M);
```

% 生成 blackman 窗

```
w_bla=(blackman(M))';
h=hd.*w_bla
```

% 频域图像的绘制

```
freqz(h,[1])

figure(2);
subplot(2,2,1),stem(n,hd);title('ideal impulse response')
axis([0 M-1 -0.4 0.5]);xlabel('n');ylabel('hd(n)')
xa=0.*n;
hold on
plot(n,xa,'k');
hold off

subplot(2,2,2),stem(n,w_bla);title('blackman window')
axis([0 M-1 0 1.1]);xlabel('n');ylabel('w(n)')

subplot(2,2,3),stem(n,h);title('actual impulse response')
```

```
axis([0 M-1 -0.4 0.5]);xlabel('n');ylabel('h(n)')
```

```
hold on
```

```
plot(n,xa,'k');
```

```
hold off
```

其响应波形如图 7-3 所示。

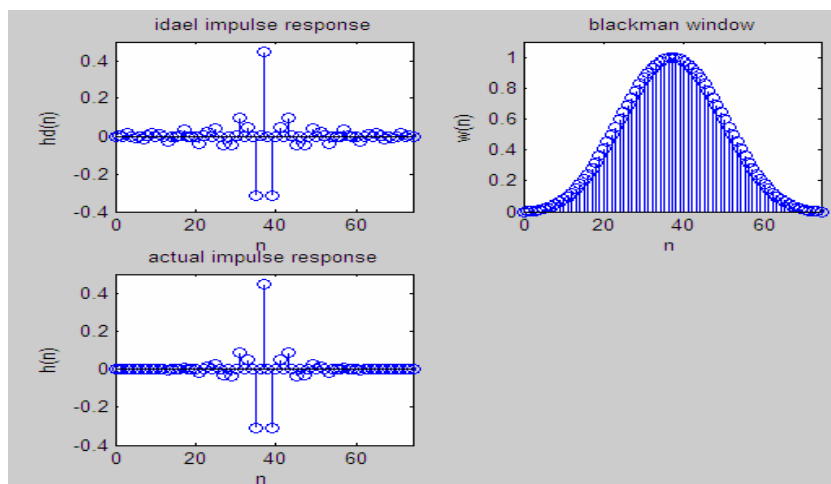
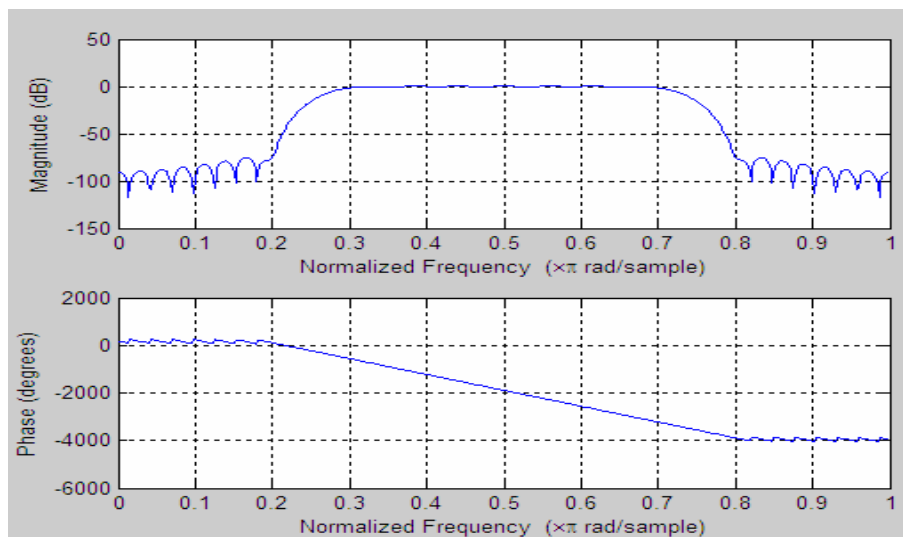


图 7-3 blackman 窗函数设计 FIR 滤波器